École Nationale Supérieure de l'Intelligence Artificielle et Sciences des Données

INTERNSHIP REPORT

FactoryFlow: A Mobile Platform for Intelligent Industrial Maintenance KPI Collection and Automated Reporting

Presented by

Badr-Eddine ESSALEHY

Program:	Management and Governance of Information Systems
Host organization:	Alf Mabrouk Nutrition Animale
Academic supervisor:	To be confirmed
Company supervisor:	To be confirmed

Academic Year 2025–2026

DEDICATION

ACKNOWLEDGEMENTS

ABSTRACT

CONTENTS

LIST OF FIGURES

LIST OF TABLES

GENERAL INTRODUCTION

CHAPTER 1

INDUSTRIAL CONTEXT AND PROBLEM DEFINITION

I. Introduction

Digital transformation in industrial environments extends beyond the automation of production equipment. It also concerns the way operational information is collected, structured, validated, analysed, and transformed into useful knowledge. Maintenance activities generate numerous indicators required for equipment monitoring, resource-consumption follow-up, anomaly identification, and periodic reporting. The value of these indicators, however, depends directly on the reliability and traceability of the underlying data.

During the internship at **Alf Mabrouk Nutrition Animale**, an important limitation was identified in the maintenance reporting process. Several **Key Performance Indicator (KPI)** values were communicated through heterogeneous WhatsApp messages and later transferred manually into reporting files. Although this communication channel enabled rapid information exchange, the absence of a standardized acquisition process introduced additional interpretation and transcription work. The existing workflow is presented in **Section IV**.

The internship project therefore focused on transforming this fragmented process into a structured and traceable digital workflow. This led to the development of **FactoryFlow**, a mobile platform dedicated to intelligent KPI acquisition, human-controlled validation, centralized storage, automated reporting, and maintenance-oriented analysis. The project objectives are detailed in **Section VIII**.

A complementary web application dedicated to dosage deviation analysis was also developed to address another industrial need related to production monitoring. While **FactoryFlow** focuses on trustworthy maintenance information and decision support, the **Dosage Analysis Web Application** focuses on detecting and monitoring deviations between expected and measured production quantities. Its role within the internship is introduced in **Section X**.

This chapter presents the industrial context, the existing maintenance workflow, its limitations, the associated data-quality challenges, and the objectives that shaped the proposed solutions.

II. Host Organization and Industrial Context

Alf Mabrouk, also known as La Marocaine d'Aliments Composés, is a Moroccan company operating in the **animal nutrition sector**. Established in 2012 and located in the **industrial zone**

of Bouznika, the company specializes in the manufacture and commercialization of compound feed intended for livestock and poultry.

Alf Mabrouk belongs to the poultry-related activities of **Cap Holding**, a Moroccan industrial and investment group active in cereal-related activities, industrial milling, and agricultural raw-material trading. This affiliation places the company within a broader industrial ecosystem involving the transformation, storage, and management of agricultural raw materials.

From an operational perspective, the technical organization is structured around complementary functions including maintenance, production, and raw-material reception and transfer. These activities are supported by supervisors, team leaders, operators, and shift-based teams. **Figure 1.1** provides a simplified representation of this organization.

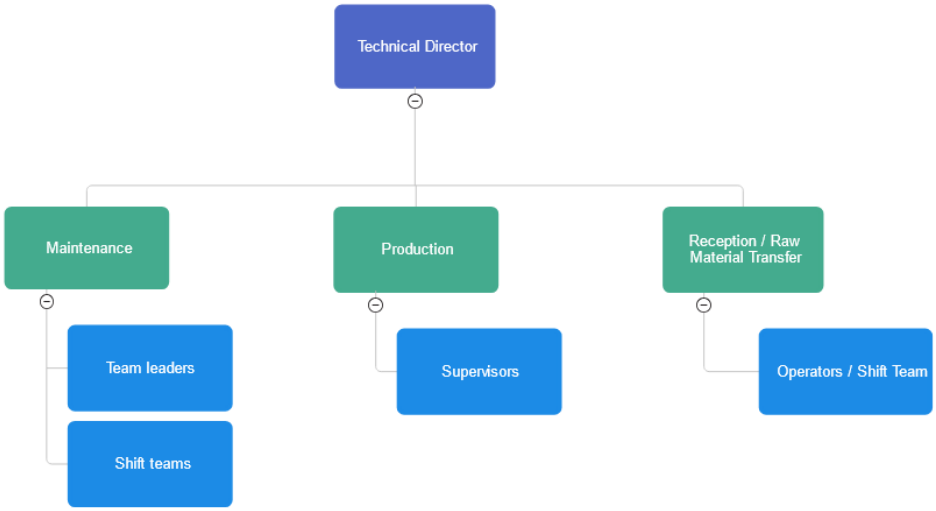

FIGURE 1.1 *Simplified technical organizational structure of Alf Mabrouk*

Production depends on the controlled transformation and combination of several raw materials and premixes. Each production recipe specifies the required ingredients and their target quantities.

The process begins with raw-material storage in dedicated silos. Ingredients are transferred to weighing systems according to the selected recipe before passing through milling. Smaller and more sensitive components are introduced through micro-dosing systems using precise weighing equipment. Liquid ingredients such as water and vegetable oils are also measured and injected before mixing. The resulting product may then pass through pressing lines, finished-product silos, and finally bulk handling or bagging. These principal stages are illustrated in **Figure 1.2**.

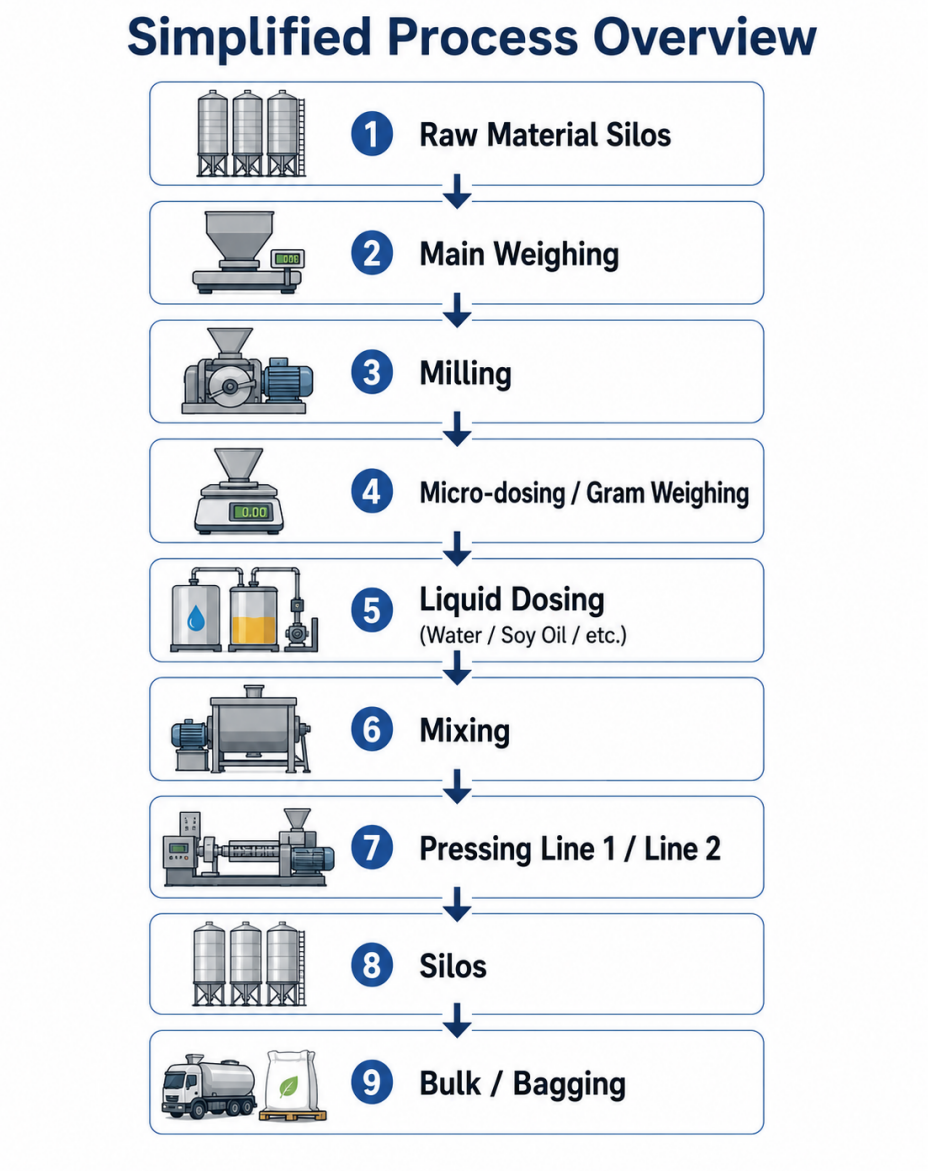

FIGURE 1.2 *Simplified overview of the compound-feed production process at Alf Mabrouk*

This production environment combines physical equipment with industrial information systems. Programmable logic controllers participate in equipment control and measurement, while recipes and operational information are managed digitally. Reliable industrial monitoring therefore depends not only on equipment operation, but also on the quality of the data collected from these activities. This relationship provides the context for the maintenance processes discussed in **Section III**.

III. Maintenance Context

Maintenance plays a central role in ensuring the availability and correct operation of the equipment involved throughout the production process. The stages shown in **Figure 1.2** depend on interconnected mechanical, electrical, electromechanical, and automated systems. A malfunction in one component may therefore influence one or several production stages.

Beyond direct interventions, the maintenance team follows operational indicators related to equipment utilisation and industrial utilities. These include, among others, operating hours, compressor measurements, water consumption, fuel quantities, oil levels, molasses levels, and other values required for technical monitoring.

As illustrated in **Figure 1.1**, maintenance activities involve different operational teams and shifts. Information collected in the field must consequently remain accessible and understandable regardless of its source or the employee who recorded it.

This context makes **data continuity and traceability** essential. A measurement may later be required for historical comparison, reporting, anomaly investigation, trend analysis, or technical decision support. The workflow previously used to transmit and structure these indicators is presented in **Section IV**.

IV. Existing KPI Collection Process

Before FactoryFlow, maintenance indicators were collected primarily through a combination of **WhatsApp communication and manual transcription**. Employees shared measurements collected during their activities or shifts through operational WhatsApp groups.

This approach was convenient for rapid human communication, but the messages were not designed for direct machine processing. Equivalent indicators could therefore appear with different spellings, abbreviations, separators, units, decimal formats, or ordering. The resulting data-quality challenges are discussed in **Section VI**.

A message could contain several indicators, only a small subset of them, or explicitly missing values. Once received, the useful information had to be manually identified, interpreted, and transferred into the reporting environment.

The original workflow is illustrated in **Figure 1.3**.

FIGURE 1.3 *Existing maintenance KPI collection and reporting workflow*

Although functional, this workflow introduced a manual transformation step between field information and the structured data used for reporting. Its principal limitations are analysed in **Section V**.

V. Limitations of the Existing Process

The existing process fulfilled the basic objective of transmitting operational information, but several limitations appeared once the messages had to be transformed into reusable data.

The first limitation concerned **manual effort**. Each message had to be interpreted before its values could be transferred into the reporting system.

A second limitation was the **risk of transcription and interpretation errors**. Decimal values could be misunderstood, information could be omitted, or a measurement could be associated with the wrong indicator.

A third limitation concerned **heterogeneous message formats**. Equivalent KPIs could be expressed using different labels, abbreviations, separators, units, and numerical representations. These variations are examined further in **Section VI**.

The process also had to preserve the distinction between **zero and missing information**. An unavailable measurement cannot safely be converted into zero because both states have different analytical meanings.

Finally, the workflow offered limited **source traceability**. Once a value had been manually copied into a reporting file, verifying its origin required returning to the original conversation.

The problem was therefore not WhatsApp itself, which remained a rapid operational communication tool. The need was to introduce a reliable processing layer between informal communication and structured industrial data. These limitations led directly to the problem formulated in **Section VII**.

VI. Data Quality Challenges

The difficulty was not limited to transferring values from one interface to another. Before a measurement could be stored reliably, the system first had to determine **what the received information represented**.

The same KPI could appear with different capitalization, accents, abbreviations, punctuation, or spacing. Numerical values could use a comma or a period as a decimal separator, while certain formats could remain ambiguous without human confirmation. Units could be attached directly to values or written separately.

Another critical situation concerned **missing measurements**. Expressions such as dashes or explicit missing-value markers had to remain distinguishable from numerical zero. FactoryFlow therefore adopts a strict rule:

Missing information is not equivalent to zero.

Messages extracted from screenshots may also contain elements unrelated to maintenance KPIs, including names, timestamps, date separators, conversation headers, and interface metadata. These elements must be separated from business information while the original source remains preserved for traceability.

Some indicators may additionally contain **multiple related measurements**. A compressor entry, for example, may contain both a principal value and an associated percentage. Such information must remain semantically linked.

Reliable acquisition therefore requires normalization, classification, numerical interpretation, KPI matching, uncertainty management, and human validation. These constraints, combined with the limitations presented in **Section V**, motivate the project problem statement.

VII. Problem Statement

As illustrated in **Figure 1.3**, the maintenance reporting process relied on manual interpretation and transcription of heterogeneous operational information. The limitations identified in **Section V** and the data-quality constraints discussed in **Section VI** reduced standardization, traceability, and analytical exploitation.

The central problem can therefore be formulated as follows:

> **How can heterogeneous maintenance KPI information be transformed into reliable, structured, traceable, and analytically valuable industrial data while preserving human control over uncertain information?**

The solution must reduce repetitive manual work while preserving the maintenance engineer's judgement. It should also move beyond data storage by transforming validated historical information into meaningful indicators, alerts, trends, and decision-support insights.

VIII. Internship Objectives

The main objective of the internship was to design and implement a mobile platform capable of transforming fragmented maintenance information into a structured, trustworthy, and exploitable industrial information flow.

More specifically, the project pursued the following objectives:

1. **Centralize maintenance KPI information** within a single mobile platform.
2. Support both **manual acquisition and image-based acquisition**, particularly from WhatsApp screenshots.
3. Use **Optical Character Recognition (OCR)** to extract textual information from imported images.
4. Process heterogeneous KPI messages through **deterministic parsing and normalization rules**.
5. Distinguish between **confirmed measurements, missing information, ambiguous values, noise, and unrecognized indicators**.
6. Preserve the original acquired content to ensure **end-to-end traceability**.
7. Maintain **human validation** before uncertain information enters the confirmed dataset.

8. Persist validated information within a **centralized database**.
9. Provide **historical consultation and interactive statistical analysis** based exclusively on validated data.
10. Introduce a **Maintenance Intelligence layer** capable of analysing historical KPI behaviour and identifying unusual patterns.
11. Apply **anomaly detection** to highlight measurements that deviate significantly from expected historical behaviour.
12. Provide **trend analysis and consumption forecasting** to support anticipation and technical decision-making.
13. Generate **contextual alerts and notifications** for abnormal measurements, relevant thresholds, missing information, and automated reporting events.
14. Generate structured **PDF and Excel reports** directly from validated maintenance information.
15. Support **scheduled report generation, email distribution, and document sharing**.

These objectives address the weaknesses identified in **Section V** while respecting the data-integrity constraints presented in **Section VI**.

IX. FactoryFlow Positioning

FactoryFlow is positioned between raw operational communication and reliable industrial decision support. It does not replace the communication habits of the maintenance team; instead, it provides a controlled environment in which information can be acquired, interpreted, validated, stored, analysed, and reported.

The platform combines **OCR-assisted acquisition, deterministic parsing, source-line classification, KPI matching, human validation, centralized persistence, historical analysis, automated reporting, and maintenance intelligence**. This architecture transforms informal operational information into a structured information asset.

A fundamental principle of the platform is:

Automation + Human Validation + Traceability = Trustworthy Industrial Reporting

Automation is therefore used where the evidence is sufficient, while ambiguous information remains under human control. Once validated, historical data becomes the foundation for statistics, anomaly detection, forecasting, and contextual alerts.

FactoryFlow consequently evolves beyond a conventional data-entry application. It acts as a **traceable industrial information pipeline** connecting field observations with reporting and maintenance-oriented decision support.

X. Complementary Dosage Analysis Project

In parallel with FactoryFlow, the **Dosage Analysis Web Application** was developed to address a production-related need concerning dosage deviations.

Production recipes define target quantities for the ingredients required by each batch. During execution, measured quantities can be compared with these targets and with the tolerance associated with each ingredient. The application centralizes these measurements and supports the detection, consultation, and follow-up of dosage anomalies.

The two projects therefore address complementary industrial problems. **FactoryFlow** focuses on transforming heterogeneous maintenance information into reliable and analytically exploitable data, whereas **Dosage Analysis** focuses on analysing deviations within the production process.

Their technical implementation and the relationship between both solutions are developed later in the dedicated Dosage Analysis chapter.

XI. Conclusion

This chapter established the industrial context and identified the weaknesses of the previous maintenance reporting workflow. Manual interpretation, heterogeneous formats, ambiguity, missing information, and limited traceability created the need for a more reliable information process.

These findings led to the development of **FactoryFlow**, combining assisted acquisition, deterministic interpretation, human validation, centralized persistence, automated reporting, and maintenance intelligence. The complementary Dosage Analysis project addresses a second industrial challenge related to production dosage deviations.

The next chapter translates these industrial needs into the **functional requirements, non-functional requirements, business rules, and system specifications** that guided the design of the proposed solutions.

CHAPTER 2

REQUIREMENTS ANALYSIS AND SYSTEM SPECIFICATION

I. Introduction

The industrial analysis presented in **Chapter 1** established that the main challenge was not the absence of maintenance data, but the way this information was acquired, interpreted, validated, and reused. Heterogeneous operational messages had to be transformed into reliable industrial information without losing their meaning or origin.

This chapter translates that need into the functional and quality requirements of **FactoryFlow**. Particular attention is given to the mechanisms that protect the value of the collected data: **human-controlled validation**, source traceability, explicit management of missing information, deterministic interpretation, and controlled automation.

The chapter also defines the business rules that govern the information lifecycle, the conditions used to validate the main workflows, and the use case model representing the interactions between the Maintenance Engineer and the platform.

II. Stakeholders and System Actors

FactoryFlow was designed for a focused industrial context rather than for a large public user base. Its principal human actor is the **Maintenance Engineer**, who interacts directly with the mobile application throughout acquisition, review, validation, consultation, analysis, and reporting.

The information produced by the platform also supports **Technical Management**, particularly through consolidated reports, historical indicators, statistics, alerts, and analytical results used for technical follow-up and decision-making.

TABLE 2.1 *Main FactoryFlow stakeholders and responsibilities*

Stakeholder	Main responsibilities and interests
Maintenance Engineer	Acquires maintenance information, reviews interpreted measurements, resolves uncertain data, confirms reports, consults KPI history and statistics, and uses the resulting information for maintenance follow-up.
Technical Management	Uses confirmed and consolidated maintenance information, generated reports, statistics, alerts, and analytical indicators to support technical supervision and decision-making.

FactoryFlow has one principal human actor, while Technical Management benefits from the validated information produced by the platform.

Several operations are performed automatically by the platform rather than initiated manually at execution time. These include **scheduled report generation**, document distribution, notification creation, and the analytical processing associated with Maintenance Intelligence. They represent system responsibilities rather than additional human actors.

III. Functional Requirements

The functional requirements describe the services required to transform raw maintenance information into **validated, traceable, and exploitable industrial data**. They cover the complete lifecycle of an observation, from its acquisition to reporting and analytical use.

1. **Authentication and secure access**

   Access to protected FactoryFlow functions requires authentication. The application maintains the user session through token-based authentication and clearly distinguishes an expired or invalid session from an ordinary application or communication error.

2. **Maintenance KPI acquisition**

   FactoryFlow supports the different acquisition situations encountered in the maintenance workflow described in **Section IV**.

   The Maintenance Engineer can:

   2.1. paste the content of a maintenance message;

   2.2. import an image or WhatsApp screenshot;

   2.3. extract visible text from an imported image using OCR;

   2.4. enter measurements manually;

   2.5. review the acquired source before validation.

These alternatives allow the platform to integrate with existing field practices without forcing a single method of data entry.

3. **Deterministic interpretation and classification**

   Acquired text is transformed into structured KPI candidates through **deterministic processing**. Normalization handles relevant variations in capitalization, accents, punctuation, spacing, numerical separators, known units, approved aliases, and recognized OCR artefacts.

   The interpretation stage keeps the following states distinguishable: recognized measurements, ambiguous values, explicit missing information, duplicate observations, unresolved KPI-like content, and safely identified source noise.

   This separation is essential because information with different semantic meaning must not follow the same validation path.

4. **Human-controlled review and KPI management**

   Before uncertain information can enter the confirmed dataset, FactoryFlow provides a dedicated **Review workflow**. The user can validate an acceptable measurement, correct a value, confirm a legitimate duplicate, preserve a missing value as **Non renseigné**, associate an unresolved line with an existing KPI, or create a new KPI when required.

   Source metadata and safely classified noise can be ignored without removing their trace from the original acquisition. Incomplete work can also be saved as a draft and resumed later.

   KPI association follows a controlled hierarchy based on canonical labels, approved aliases, and fuzzy suggestions. **Fuzzy matching remains advisory**: a weak suggestion can assist the user, but it cannot confirm an association or prevent the creation of a genuinely new KPI.

   A reusable association is memorized only after an explicit user decision.

5. **Confirmation, persistence, and traceability**

   A report becomes part of the trusted dataset only after the required review decisions have been completed. **Confirmation remains blocked while mandatory human resolution is still pending**.

   For confirmed reports, FactoryFlow preserves the information required to understand both the measurement and its origin. This includes the original source, confirmed KPI values, explicit missing values, legitimate duplicate observations, relevant ignored lines, and the decisions taken during Review.

   The resulting history therefore records not only a final value, but also the context needed to explain how that value entered the system.

6. **Historical consultation and statistics**

   Confirmed maintenance information can be consulted over time through the report history and statistical views. The engineer can filter relevant information, inspect KPI evolution, consult statistical summaries, and interact with historical trend visualizations.

   Statistical calculations rely on **confirmed measurements**. Missing information remains absent from numerical calculations rather than being silently converted into zero.

7. **PDF and Excel reporting**

   FactoryFlow transforms validated maintenance information into structured **PDF and Excel documents**.

   Two reporting scopes are supported:

   - an **individual export** contains only the selected confirmed maintenance report;
   - a **consolidated export** brings together the confirmed reports belonging to a daily, weekly, monthly, or custom period.

   Generated documents preserve missing measurements as **Non renseigné**, retain the relevant reporting provenance, and remain available for later consultation, download, or sharing.

8. **Scheduling, distribution, and notifications**

   Repetitive reporting tasks can be automated through configurable planifications. The engineer can create daily, weekly, or monthly schedules, select the execution time and document formats, enable or pause a schedule, and optionally configure automatic email distribution.

   At execution time, the platform determines the appropriate reporting period, generates the requested documents, stores the result, performs the configured distribution, and records the execution outcome.

   Notifications keep the engineer informed when a scheduled document is ready or when generation or delivery requires attention. A distribution failure does not invalidate a document that was generated successfully.

9. **Maintenance Intelligence**

   Validated historical information provides the foundation for a higher analytical layer dedicated to maintenance follow-up.

   This layer supports:

   9.1. detection of abnormal KPI behaviour and significant deviations;

   9.2. analysis of historical trends;

9.3. forecasting of suitable consumption or maintenance-related indicators when sufficient historical data is available;

9.4. presentation of analytical findings in an understandable form;

9.5. contextual alerts when relevant conditions are identified.

The essential constraint is that Maintenance Intelligence operates on **validated historical data**, not directly on unreviewed OCR or parser output.

Together, these functions define FactoryFlow as more than a data-entry application. The platform establishes a controlled information chain from field acquisition to validated reporting and decision support. The quality constraints governing this chain are defined in the following section.

IV. Non-Functional Requirements

Functional coverage alone is not sufficient in an industrial reporting context. FactoryFlow must also preserve the reliability, usability, and traceability of the information while remaining maintainable and adaptable to its deployment environment.

The principal quality requirements are summarized in **Table 2.2**.

TABLE 2.2 *Main non-functional requirements of FactoryFlow*

ID	Category	Requirement
NFR-01	**Data Integrity**	Confirmed measurements, missing information, uncertain values, duplicates, unresolved content, and ignored metadata remain semantically distinct. Analytical functions use validated data, and missing information is never silently interpreted as zero.
NFR-02	**Reliability and Resilience**	Validation, draft saving, confirmation, reporting, and scheduled execution must produce predictable results. A temporary communication or email failure must not silently corrupt confirmed information or invalidate a successfully generated document.
NFR-03	**Traceability**	Information acquired through an image, pasted text, or manual entry remains traceable to its source throughout interpretation, review, persistence, and report generation.

Table 2.2 – continued

ID	Category	Requirement
NFR-04	**Usability**	The mobile workflow clearly distinguishes ready, missing, uncertain, duplicate, and unresolved information and provides an understandable action whenever user intervention is required.
NFR-05	**Performance**	Navigation, Review interactions, history consultation, and statistical visualization remain responsive on the target Android devices without unnecessary blocking of the user interface.
NFR-06	**Security**	Protected resources require authentication. Credentials, tokens, database connections, and application secrets are handled securely and remain outside exposed application logic.
NFR-07	**Maintainability and Extensibility**	Acquisition, interpretation, validation, persistence, reporting, scheduling, and analytical responsibilities remain sufficiently separated to evolve independently. New KPIs, aliases, interpretation rules, or analytical mechanisms can be introduced without redesigning the complete platform.
NFR-08	**Interoperability and Deployment**	The Android application, Spring backend, PostgreSQL database, OCR runtime, email service, and generated documents communicate through defined interfaces. Environment-dependent endpoints, storage paths, connections, and secrets remain externally configurable.

These requirements reflect a central design principle of FactoryFlow: **automation must reduce manual effort without weakening the meaning, reliability, or traceability of industrial information**. The business rules below translate this principle into concrete safeguards.

V. Business Rules

Business rules protect the semantic integrity of the information handled by FactoryFlow. Unlike interface details, they remain valid regardless of the screen or technical component performing the operation.

Rather than describing every interface action separately, **Table 2.3** concentrates the rules that directly protect industrial data and reporting behaviour.

TABLE 2.3 *Business rules governing FactoryFlow data processing*

ID	Rule	Description
BR-01	**Missing and zero remain distinct**	An explicitly missing measurement remains **Non renseigné**; it is never converted automatically into zero. Conversely, an explicit numerical zero remains a valid measurement.
BR-02	**Interpretation is deterministic**	Normalization and KPI interpretation follow deterministic rules. When more than one credible interpretation remains possible, the uncertainty is kept visible and requires human review rather than an automatic guess.
BR-03	**KPI association follows a controlled priority**	Association gives priority to an exact canonical label, followed by an approved alias. Fuzzy matching is used only as an advisory mechanism when no higher-authority match exists and cannot confirm a weak association on its own.
BR-04	**Learning and KPI creation are explicit**	New aliases, reusable associations, and new KPI definitions require an explicit user decision. FactoryFlow does not silently learn from an uncertain suggestion, and the creation of a new KPI checks for an equivalent existing definition.
BR-05	**Uncertainty must be resolved before confirmation**	Ambiguous measurements, review-required duplicates, corrected missing values, and unresolved KPI associations require an explicit resolution before the report can enter the confirmed dataset.

Table 2.3 – continued

ID	Rule	Description
BR-06	**Duplicate observations remain independent**	Legitimate occurrences of the same KPI are not silently merged or overwritten. Each occurrence can be reviewed independently and retains its own source traceability.
BR-07	**Composite measurements remain linked**	When one source observation contains several related measurements, such as a main compressor value and an associated percentage, those values remain linked to the same observation.
BR-08	**Safe noise remains distinct from unresolved information**	Deterministically recognized metadata, timestamps, interface fragments, and other non-business elements may be treated as safe noise. Bulk-ignore actions apply only to this category; uncertain KPI-like information remains visible until resolved.
BR-09	**The original source is preserved**	Raw acquired content remains available after OCR extraction, normalization, association, review, and confirmation. A validated measurement can therefore be traced back to the information from which it originated.
BR-10	**Validated information drives analysis and reporting**	Statistics, trend analysis, Maintenance Intelligence, PDF generation, and Excel generation rely on **confirmed information**. Missing observations are excluded from numerical calculations unless an analytical method explicitly supports them.
BR-11	**Reporting scope and periods are explicit**	An **individual export** contains only the selected confirmed source report. A **consolidated export** includes the confirmed reports belonging to the selected daily, weekly, monthly, or custom period. Weekly periods follow Monday to Sunday, monthly periods follow complete calendar months, and scheduled generation uses the corresponding completed reporting period.

These rules form the semantic safeguards between raw acquisition and trustworthy industrial information. Their purpose is not to prevent automation, but to ensure that automation never silently changes the meaning of the original data.

This principle can be summarized as:

Automation + Human Validation + Traceability = Trustworthy Industrial Reporting

VI. Acceptance Criteria

The acceptance criteria provide a concise set of observable conditions against which the main FactoryFlow workflows can be evaluated. They intentionally focus on complete business scenarios rather than repeating every individual requirement.

TABLE 2.4 *Main acceptance criteria for FactoryFlow*

ID	Area	Acceptance criterion
AC-01	**Acquisition and interpretation**	Maintenance information can be acquired through pasted text, image import, or manual entry. The interpretation stage distinguishes recognized measurements from missing, ambiguous, duplicate, unresolved, and safely classified non-business information.
AC-02	**Data integrity and Review**	A missing measurement remains **Non renseigné**, while a real zero remains numerical zero. Any element requiring human review exposes an explicit resolution path before it can become confirmed data.
AC-03	**KPI association and noise handling**	An unresolved KPI can be associated with an existing definition or explicitly created as a new indicator. Weak fuzzy suggestions remain optional, while bulk-ignore operations cannot discard unresolved KPI-like information.
AC-04	**Traceability and confirmation**	A report cannot be confirmed while mandatory resolutions remain pending. After confirmation, its measurements remain traceable to the acquired source and incomplete work can be recovered through the draft workflow before confirmation.

Table 2.4 – continued

ID	Area	Acceptance criterion
AC-05	**History and statistics**	Confirmed measurements can be consulted historically and used for statistical summaries and interactive trend visualization. Missing observations do not enter ordinary numerical calculations.
AC-06	**Reporting**	A confirmed report can be exported individually, while confirmed reports can also be consolidated over daily, weekly, monthly, or custom periods. PDF and Excel outputs preserve **Non renseigné** values and the relevant report provenance.
AC-07	**Scheduling and distribution**	Configured schedules can generate the requested reporting formats for the correct period and optionally distribute them by email. Generation and delivery outcomes remain visible through execution history and notifications, and an email failure does not remove a successfully generated document.
AC-08	**Maintenance Intelligence**	Anomaly detection, trend analysis, forecasting, and contextual alerts operate on validated historical measurements rather than raw OCR or unreviewed parser output.

These criteria establish the bridge between specification and validation. The testing chapter will later evaluate the implemented workflows against these expected behaviours rather than against isolated interface details.

VII. Use Case Modeling

The functional perimeter defined above is summarized through a UML use case model. FactoryFlow has one principal human actor: the **Maintenance Engineer**.

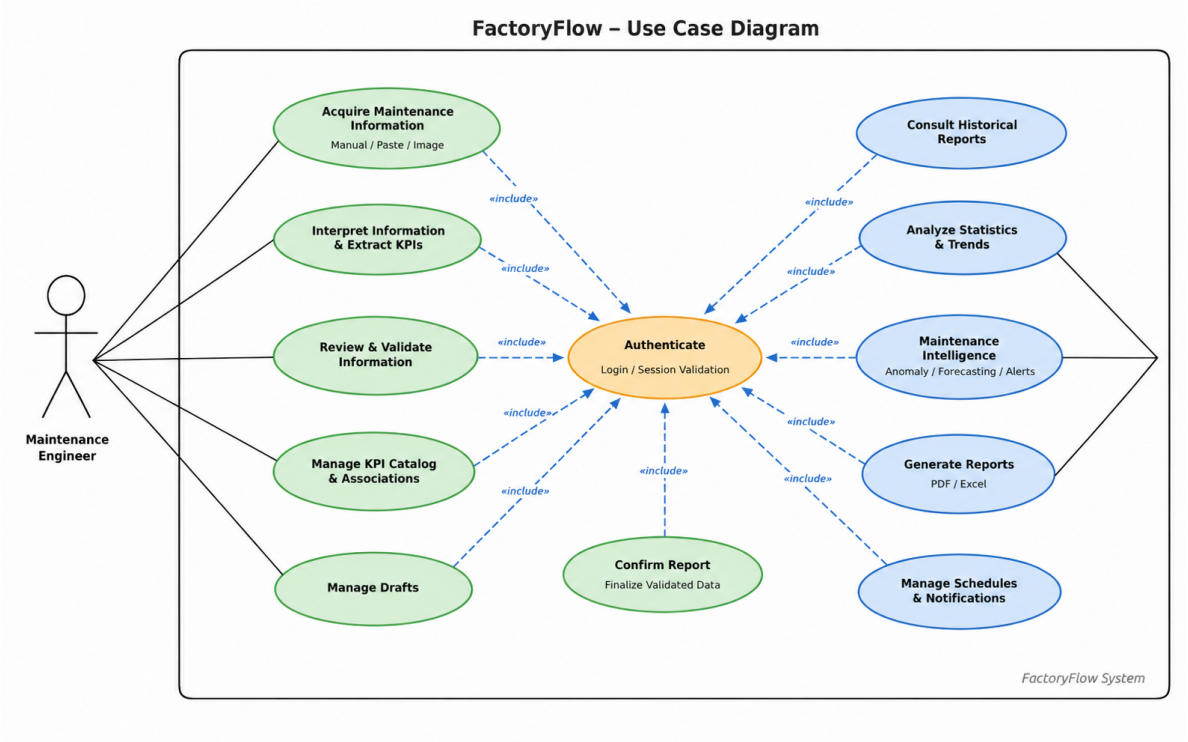

FIGURE 2.1 *FactoryFlow use case diagram*

As shown in **Figure 2.1**, authentication precedes access to the protected functions of the platform.

The Maintenance Engineer can acquire operational information, review and validate interpreted measurements, manage KPI associations, preserve incomplete work as drafts, and confirm trusted reports. Once information is confirmed, the same actor can consult its history, analyse KPI evolution, generate individual or consolidated documents, and manage recurring reporting schedules.

The model also reflects the analytical role of FactoryFlow. Validated historical measurements feed statistics and, at a higher level, Maintenance Intelligence functions. This keeps a clear separation between **raw acquisition**, **human validation**, and **analytical exploitation**.

VIII. Conclusion

This chapter transformed the industrial needs identified in **Chapter 1** into a concise specification of FactoryFlow.

The analysis identified the **Maintenance Engineer** as the principal human actor and defined the functions required to acquire heterogeneous maintenance information, interpret it deterministically, resolve uncertainty, preserve traceability, build a trusted history, and transform confirmed data into statistics and professional reports.

The specification also established the quality and business safeguards that give these functions their industrial value. In particular, **missing information remains distinct from zero**,

fuzzy matching never replaces explicit validation, duplicate observations remain traceable, and analytical processing relies on confirmed historical measurements.

Finally, the reporting and automation requirements extend the same principle to document generation and distribution: individual and consolidated scopes remain explicit, reporting periods are deterministic, and operational failures remain visible instead of silently compromising the resulting information.

With the expected behaviour now established, **Chapter 3** presents the software architecture and design choices used to translate these requirements into the technical components of FactoryFlow.

CHAPTER 3

SYSTEM DESIGN AND ARCHITECTURE

I. Introduction

The previous chapter defined what **FactoryFlow** must achieve and the rules that protect the integrity of the maintenance information it processes. The next step is to explain how these requirements were translated into a coherent software architecture.

FactoryFlow was designed as a set of **clearly separated responsibilities**. The Android application manages user interaction and field acquisition, the backend coordinates business logic and secure access to data, **the OCR runtime** extracts text from imported images, and **PostgreSQL** provides persistent storage for validated information. Around this core, dedicated components support reporting, scheduling, notifications, and analytical processing.

This separation is particularly important because the platform does not treat acquisition, interpretation, and validation as a single operation. Information moves through a controlled sequence in which **raw input is preserved**, deterministic processing structures the data, uncertain elements remain under **human control**, and only confirmed information becomes available for reporting and analysis.

This chapter presents the architecture from the outside inward. It first introduces the **global system architecture** and the communication between the main components. It then examines the Android application, backend architecture, persistence, security, OCR and deterministic interpretation, reporting and scheduling, Maintenance Intelligence, deployment and integration, and finally the principal UML class relationships that support these architectural responsibilities.

II. Global System Architecture

FactoryFlow follows a **modular client–server architecture** in which the main responsibilities of the platform are deliberately separated. The Android application manages field interaction, the backend coordinates business processing, the OCR runtime extracts text from imported images, PostgreSQL stores persistent information, and dedicated services support reporting, automation, and analytical processing. This separation makes the system easier to understand, maintain, and extend while preserving the integrity of the maintenance information lifecycle.

Figure 3.1 presents the **high-level architectural map** of FactoryFlow. It summarizes the main subsystems and the principal information flow that connects operational acquisition, controlled validation, persistent storage, reporting, and Maintenance Intelligence.

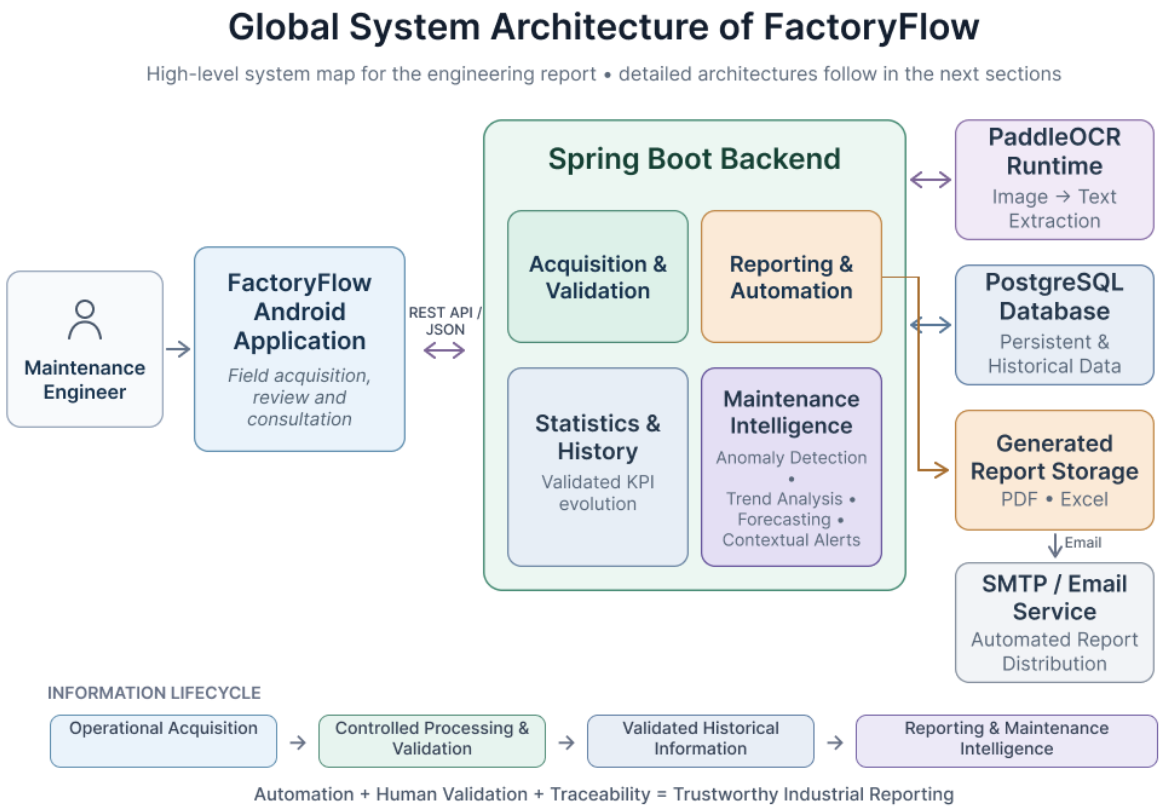

FIGURE 3.1 *Global system architecture of FactoryFlow*
Source: Created by the author using Figma (https://www.figma.com).

As shown in **Figure 3.1**, the process begins with the **Maintenance Engineer**, who interacts with the **FactoryFlow Android application** to acquire, review, and consult maintenance information. The mobile client acts as the system's field interaction layer. It supports operational acquisition and human review, but it does not communicate directly with the database or the OCR engine.

All protected communication passes through the **Spring Boot backend** using a **REST API over JSON and HTTPS**. At this level, the backend acts as the **central coordination layer** of FactoryFlow. It manages business workflows, secure access, human-validation orchestration, and the integration of the other technical services shown in the figure.

For image-based acquisition, the backend communicates with a separate **PaddleOCR runtime**. This component is responsible only for **text extraction**. Once text has been extracted, FactoryFlow continues the processing through its own deterministic interpretation logic, including parsing, KPI matching, and review preparation. This distinction is essential: **OCR extraction and business interpretation remain two different responsibilities**.

Validated and persistent information is stored in **PostgreSQL**, which forms the historical data foundation of the platform. From this validated base, FactoryFlow supports two major exploitation paths. The first is **Reporting and Automation**, which covers PDF and Excel generation, scheduling, notification support, generated-report storage, and optional email distribution through the external **SMTP / Email Service**. The second is **Maintenance Intelligence**, where validated historical KPI data feeds anomaly detection, trend analysis, forecasting, and contextual alerts.

A central architectural principle visible in Figure 3.1 is the separation between **raw acquisition**, **deterministic interpretation**, **human validation**, and **analytical exploitation**. Unreviewed OCR or parser output is never treated as trusted historical information. Only **validated data** is allowed to feed reporting, statistics, and Maintenance Intelligence.

This high-level figure serves as the architectural entry point for the remainder of the chapter. The next section focuses on the Android application and progressively refines the client-side organization introduced here.

III. Android Application Architecture

Within FactoryFlow, the Android application forms the **interaction layer** between the Maintenance Engineer and the services coordinated by the backend. It supports the complete user-facing maintenance workflow, from operational acquisition and review to historical consultation, reporting, statistics, notifications, and scheduling.

These functions belong to different stages of the maintenance-information lifecycle. The mobile client is therefore structured so that **acquisition, validation, consultation, and reporting** remain clearly separated while sharing a consistent state and communication model. This organization prevents individual screens from becoming tightly coupled and allows each functional area to evolve without disrupting the rest of the application.

The Android client remains focused on **user interaction, state representation, workflow coordination, and secure service access**. Business responsibilities such as deterministic interpretation, persistent validation, report generation, scheduling execution, and analytical processing remain coordinated by the backend. This separation keeps the mobile layer centred on presenting information and guiding the Maintenance Engineer through controlled workflows rather than duplicating server-side logic.

The following subsections first examine the **internal functional organization** of the Android application and then progressively introduce the architectural patterns and technologies that support its operation.

III.1. Internal Application Architecture

The FactoryFlow Android application is organized around the **lifecycle of maintenance information** rather than around isolated screens. Its functional areas guide the Maintenance Engineer

from secure access and acquisition to human validation, confirmation, and the subsequent consultation and exploitation of validated information.

Figure 3.2 presents this functional organization and shows how the main Android capabilities participate in the same controlled workflow.

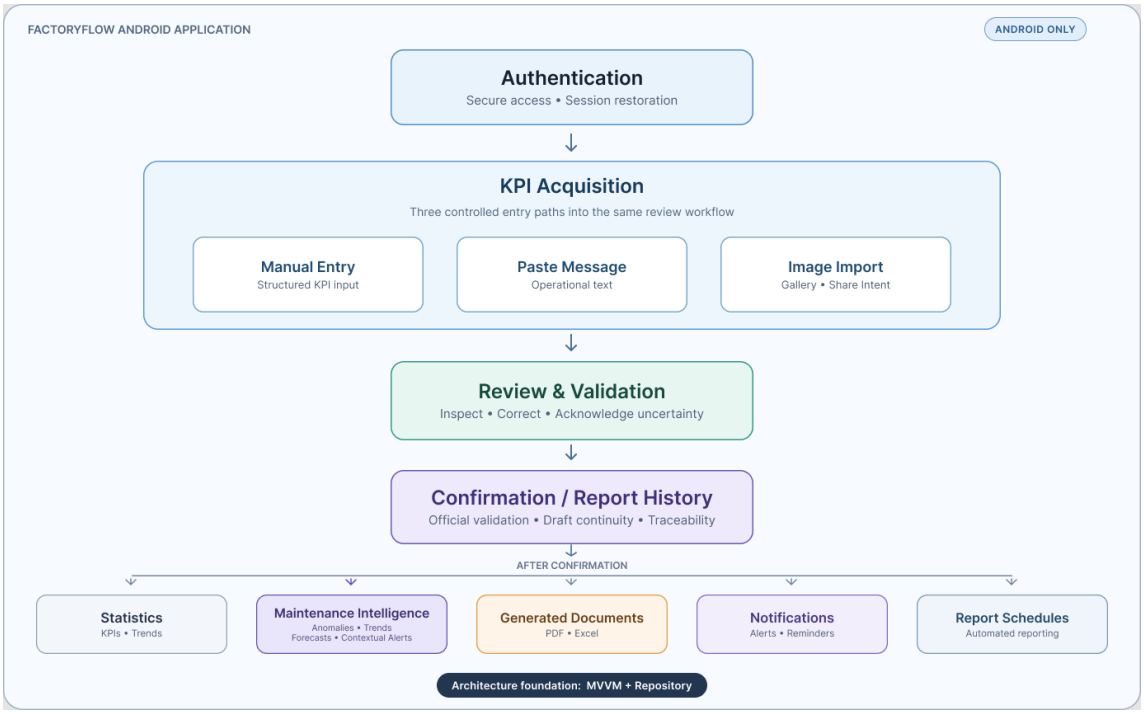

FIGURE 3.2 *Internal functional architecture of the FactoryFlow Android application*
Source: Created by the author using Figma (figma.com).

As illustrated in **Figure 3.2**, authenticated access leads to three complementary acquisition paths: **Manual Entry**, **Paste Message**, and **Image Import**. Although these paths introduce information in different forms, they all converge toward the same **Review & Validation** stage. The acquisition method therefore does not change the level of control applied before information can be trusted.

Review forms the principal human-control point of the mobile workflow. The Maintenance Engineer inspects the structured interpretation prepared by the processing pipeline, corrects detected values when necessary, and explicitly resolves elements that require human judgment. Confirmation then marks the transition from information under review to **validated historical information**, while draft continuity and traceability preserve the context in which that information was acquired and validated.

After confirmation, validated information becomes available to the application's consultation and exploitation functions. These include **Statistics**, **Maintenance Intelligence**, generated PDF and Excel documents, notifications, and recurring report schedules. Maintenance Intelligence exposes anomaly detection, trend analysis, forecasting, and contextual alerts derived from validated historical measurements; the mechanisms behind these analytical capabilities are examined later in their dedicated architecture section.

The Android workflow can therefore be summarized as *Authenticate → Acquire → Review → Confirm → Consult and Exploit*. Supporting these functional areas requires a consistent mechanism for managing state, user actions, and data access. This role is provided by the **MVVM and Repository pattern**, introduced next.

III.2. MVVM and Repository Pattern

FactoryFlow organizes its Android client around the **Model–View–ViewModel (MVVM) pattern**, complemented by a **Repository layer**. This architecture separates interface responsibilities from state management and remote data access, preventing screens from directly coordinating network operations or backend-facing data logic. **Figure 3.3** summarizes these relationships.

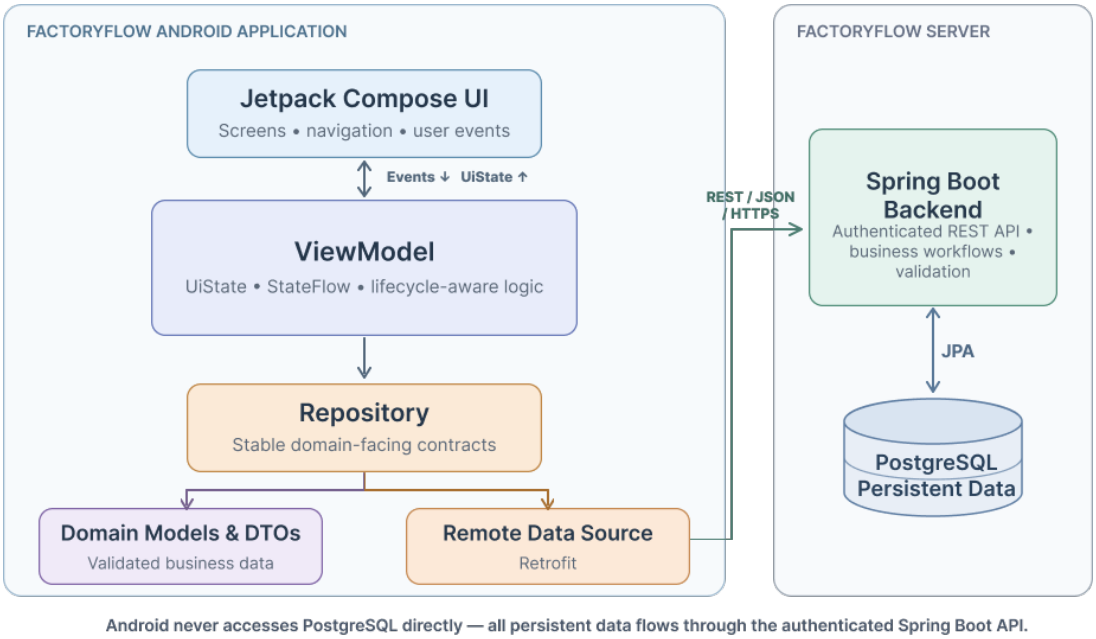

FIGURE 3.3 *MVVM and Repository architecture of the FactoryFlow Android application*
Source: Created by the author using Figma (figma.com).

At the presentation level, the **Jetpack Compose UI** displays application state and forwards user actions to the corresponding **ViewModel**. Each ViewModel exposes a structured **UiState**, propagated through **StateFlow**, so that screens react consistently to loaded information, validation states, progress indicators, and operation results. This keeps visual components focused on presentation while the ViewModel coordinates their state.

When data access is required, the ViewModel delegates operations through the **Repository**. Repositories provide stable domain-facing operations and isolate the presentation layer from the technical details of remote communication. Domain models and **DTOs** represent the information exchanged across these layers, while the **Remote Data Source** uses **Retrofit** to communicate with the authenticated Spring Boot REST API.

This separation establishes an important FactoryFlow boundary: the Android application **never accesses PostgreSQL directly**. Persistent information and server-side business processing remain behind the backend API, while responses return through the Repository and

ViewModel until the resulting UiState is reflected by the interface. The resulting architecture improves **maintainability, testability, and consistency** while allowing presentation, state management, and communication concerns to evolve independently.

With this structure established, the next subsection focuses on the technologies used to implement the Android presentation layer: **Kotlin and Jetpack Compose**.

III.3. Kotlin and Jetpack Compose

The FactoryFlow Android application is implemented natively in **Kotlin**. Its strong type system, concise syntax, and integration with the Android ecosystem provide a reliable basis for implementing the application's presentation logic, state handling, and interaction flows while keeping the codebase maintainable as the platform evolves.

The user interface is built with **Jetpack Compose**, using a declarative model in which the visible screen reflects the current application state. This approach is particularly suitable for FactoryFlow because several interfaces, including acquisition, Review and Validation, notifications, statistics, and Maintenance Intelligence, must react to changes in data and user decisions.

Compose also promotes the creation of **reusable UI components** and a consistent visual structure across the application. Combined with the state management provided by the MVVM architecture, it keeps the interface synchronized with the underlying *UiState* while preserving a clear separation between presentation, workflow coordination, and data access.

As several Android components depend on shared services such as repositories, API clients, and ViewModels, their creation and lifecycle must also remain consistent across the application. This responsibility is handled through **Hilt dependency injection**, introduced in the following subsection.

III.4. Dependency Injection with Hilt

FactoryFlow uses **Hilt** as its dependency injection framework. Built on top of Dagger, Hilt is the **recommended dependency injection solution for Android applications** and provides Android-aware mechanisms for creating, providing, and managing dependencies according to application lifecycles.

In FactoryFlow, several components depend on shared services such as API clients, repositories, and authentication-related services. Rather than allowing each component to construct these dependencies independently, Hilt provides them through a centralized dependency graph. This keeps object creation separate from the components that consume them and reduces unnecessary coupling across the application.

This organization reinforces the separation established by the **MVVM architecture**. ViewModels receive the repositories they require, while repositories depend on the communication services and remote data sources used to access backend functionality. Hilt also manages their lifecycles so that shared services can be reused where appropriate while screen-related ViewModels remain scoped to the interfaces that consume them.

The resulting structure improves **maintainability and testability** by making dependencies explicit and allowing concrete implementations to be replaced without tightly coupling them to presentation components. The communication services provided through this architecture interact with the FactoryFlow backend using **Retrofit and Moshi**, examined in the following subsection.

III.5. REST Communication with Retrofit and Moshi

The FactoryFlow Android application and the Spring Boot backend operate as separate software components and therefore communicate through a clearly defined service interface. FactoryFlow adopts a **REST-based communication model**, in which the mobile application sends HTTP requests to backend endpoints and receives structured responses represented primarily as JSON. This approach preserves a clear client–server boundary and allows the Android application to consume backend capabilities without depending on their internal implementation.

On the Android side, **Retrofit** provides the client abstraction used to access **the REST API**. Backend operations are represented as typed service operations and invoked through the Repository layer rather than directly from the user interface. Depending on the requested workflow, FactoryFlow uses HTTP methods such as GET, POST, PATCH, and DELETE. Protected requests also carry the authenticated session token using the Bearer authorization mechanism; the complete **JWT authentication architecture** is examined later in the dedicated security section.

Moshi complements Retrofit by managing the conversion between the JSON representations exchanged through the API and the typed data structures used within the Android application. Outgoing application data can therefore be serialized into JSON when required, while incoming responses are deserialized into objects that repositories and ViewModels can consume. This prevents presentation components from handling raw JSON and keeps communication concerns separated from interface behaviour.

Figure 3.4 illustrates how Retrofit, Moshi, and the REST interface participate in the communication between the Android application and the Spring Boot backend.

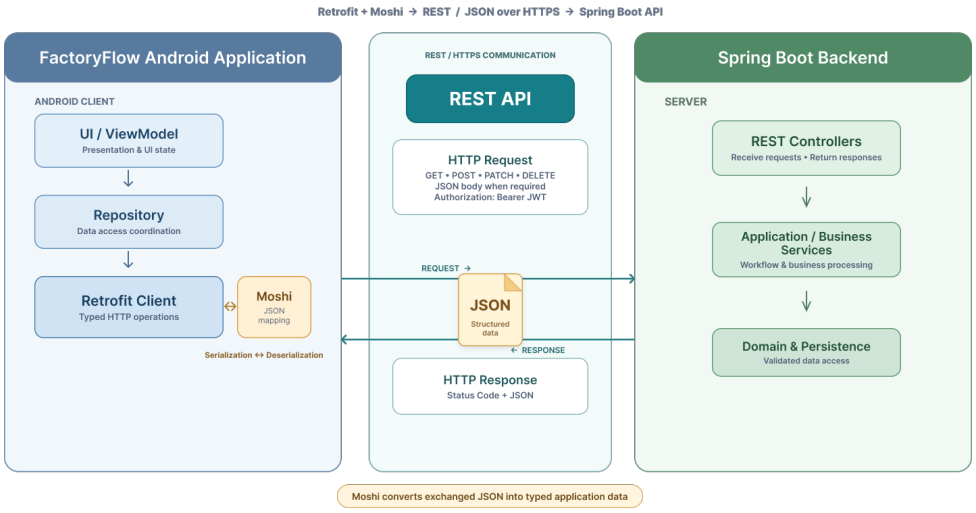

FIGURE 3.4 *REST communication between the FactoryFlow Android application and Spring Boot backend*

Source: Created by the author using Figma (figma.com).

As shown in **Figure 3.4**, a user operation is propagated from the UI and ViewModel through the Repository to the Retrofit client. The request then crosses the authenticated REST boundary as **JSON over HTTPS** and reaches the corresponding Spring Boot controller, after which the backend coordinates the required application processing. The resulting response returns with an HTTP status and, when necessary, a JSON payload that Moshi converts into typed application data before it is propagated back through the Repository and ViewModel to the interface.

Together, Retrofit and Moshi maintain a clear separation between **network communication and application behaviour**. ViewModels do not construct HTTP requests or interpret raw JSON, while repositories preserve the stable data-access boundary established by the MVVM architecture. Since these remote operations must be executed without blocking the Android interface, their asynchronous coordination relies on **Kotlin Coroutines and Flow**, examined in the following subsection.

III.6. Asynchronous Processing with Coroutines and Flow

Android interfaces must remain responsive while operations such as API requests, authentication, report retrieval, image processing requests, or analytical-data loading are executed in the background. FactoryFlow addresses this requirement using **Kotlin Coroutines**, lightweight **concurrency** components designed to execute asynchronous tasks without requiring a dedicated operating-system thread for each operation. A coroutine can suspend its execution while waiting for an external operation, such as a network response, and later resume when the result becomes available, allowing system resources to be used more efficiently.

Within FactoryFlow, **asynchronous operations** are coordinated primarily from the **ViewModel and Repository layers**. When the Maintenance Engineer triggers an action, the ViewModel initiates the corresponding coroutine and delegates the required data operation to the

Repository. While the remote request is being processed, the Android main thread remains available for interface rendering and user interaction. Once the operation completes, its result is propagated back to the ViewModel, which updates the state represented by the screen.

Kotlin **Flow** complements coroutines by representing asynchronous streams of values. FactoryFlow particularly relies on **StateFlow** to expose the current ***UiState*** from ViewModels to Jetpack Compose. As previously illustrated in **Figure 3.3**, changes in loading status, retrieved information, validation results, errors, or operation outcomes are reflected through this observable state. Compose can therefore react to state changes and update the visible interface without requiring manual synchronization between individual UI components.

The combination of Coroutines, Flow, and StateFlow reinforces the **state-driven MVVM architecture** adopted by FactoryFlow. Long-running or remote operations remain separated from visual rendering, while the interface continues to represent their progress and results consistently. This asynchronous model is particularly important for workflows involving backend communication, report operations, notifications, and Maintenance Intelligence data, where processing time may vary without affecting the responsiveness of the mobile application.

As application state must also remain coherent when the Maintenance Engineer moves between different functional areas, the next subsection examines the **Navigation Architecture** used to coordinate transitions between FactoryFlow screens.

III.7. Navigation Architecture

FactoryFlow uses **Navigation Compose** to coordinate movement between Compose destinations through a centralized navigation structure. Integrated with the declarative interface model used by the application, it provides explicit control over active destinations and navigation history. The navigation layer determines where the Maintenance Engineer is located within the application, while the state and operations associated with each destination remain managed by their corresponding ViewModels.

Navigation begins with a **session-aware application entry**. Authentication is therefore not treated as an ordinary destination alongside the functional areas of FactoryFlow. At startup, the current authentication state is evaluated before protected content is exposed. A valid session provides access to the authenticated application, whereas an unavailable or invalid session directs the user to the authentication interface. The JWT mechanisms supporting authenticated access are examined later in the dedicated **Security Architecture** section.

Figure 3.5 distinguishes the stable application destinations from the task-oriented workflows that impose a specific sequence of operations.

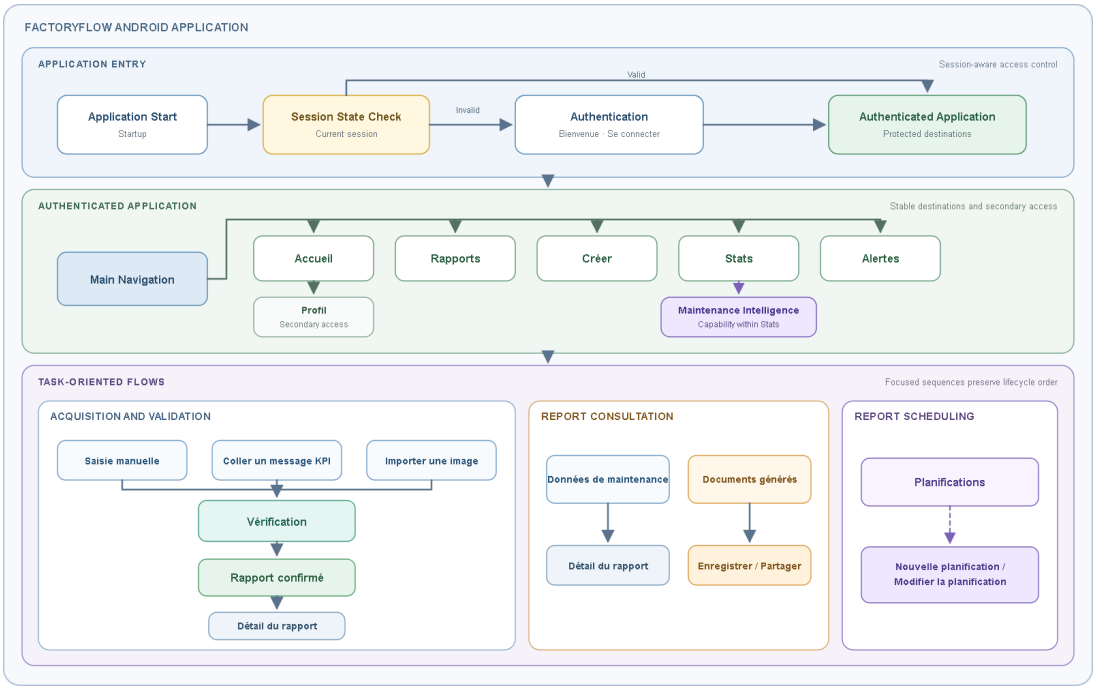

FIGURE 3.5 *Navigation architecture of the FactoryFlow Android application*
Source: Created by the author using Figma (figma.com).

As illustrated in **Figure 3.5**, the main navigation exposes *Accueil*, *Rapports*, *Créer*, *Stats*, and *Alertes*, while *Profil* remains available through secondary access. **Maintenance Intelligence** belongs to the analytical scope of *Stats*. These stable destinations differ from controlled task flows such as ***Créer*** → ***Vérification*** → ***Rapport confirmé***, where the sequence is significant because Review and confirmation protect the transition from acquired information to validated historical data.

Navigation Compose also maintains the **back stack** required to return between destinations, while application data remains outside the responsibility of the navigation layer. **Navigation determines the active destination and navigation history, whereas ViewModels remain responsible for the state and operations associated with each functional area**. Loading states, validation edits, retrieved information, and operation results therefore continue to be represented through StateFlow and structured *UiState* objects.

This separation keeps access control, workflow progression, and navigation history coherent without transferring business responsibility to the navigation mechanism. Coroutines and StateFlow preserve and propagate application state, whereas Navigation Compose determines where that state is presented as the Maintenance Engineer moves through FactoryFlow. The following subsection completes the Android architecture by synthesizing these mechanisms through the application's **end-to-end data flow**.

III.8. Android Data Flow

The complete Android-side data flow results from the mechanisms previously illustrated in **Figures 3.3–3.5**. A FactoryFlow operation begins when the Maintenance Engineer performs an

action in a Compose screen. The interface captures that intent but does not execute the operation itself. Instead, it delegates the event to the screen's ViewModel, which coordinates the interaction through the appropriate Repository. Hilt supplies the dependencies required by these components, keeping their construction separate from presentation logic and application behaviour.

The outgoing path can therefore be summarised as ***User action*** → ***Compose UI*** → ***ViewModel*** → ***Coroutine*** → ***Repository*** → ***Retrofit/Moshi*** → ***Spring Boot API***. The ViewModel launches the operation within a coroutine so that remote work does not block the main interface thread. The Repository exposes the requested operation through an application-facing abstraction, while Retrofit performs the HTTP exchange and Moshi converts between Kotlin models and the structured JSON representation exchanged with the backend.

The return path reverses these responsibilities. Retrofit and Moshi receive the backend response, the Repository maps the data-access result, and the ViewModel converts it into a stable *UiState*. StateFlow then propagates that state to Compose, which renders the corresponding loading, success, empty, or error presentation. Depending on the workflow, the backend response may result from persistence, OCR processing, report generation, or analytical processing. Android does not expose raw HTTP responses or JSON documents directly to the screen.

Navigation determines where the resulting state is presented, while each destination's ViewModel remains responsible for owning and updating that state. From the Android perspective, the **Spring Boot API** therefore constitutes the controlled boundary through which persistent data, business processing, reporting, interpretation, and analytics are accessed. Everything beyond this boundary belongs to the **backend architecture**, examined in the following section.

IV. Backend Architecture

IV.1. Java 21 and Spring Boot Foundation

FactoryFlow's backend is implemented as a **single deployable application** using **Java 21 & Spring Boot 3.5.16**. It forms the authoritative server-side boundary of the platform: the Android client collects user actions and presents application state, while the backend coordinates protected access to persistent data and business capabilities. This centralized foundation is appropriate for FactoryFlow because authentication, KPI processing, validation, reporting, scheduling, and analytical operations must follow consistent rules regardless of the Android screen initiating them.

Java 21 provides a stable long-term-support foundation for this implementation. Its strong static type system and mature tooling help detect inconsistencies during development, while its established ecosystem supports the security, persistence, document-generation, and scheduling libraries required by the platform. FactoryFlow uses **Maven** to define the Java version, manage dependency versions, execute tests, and produce a reproducible backend build.

The selected Java version is therefore an implementation baseline recorded by the build configuration rather than a permanent business constraint.

Spring Boot provides the runtime and integration foundation surrounding the application's business modules. It simplifies application startup, configuration profiles, dependency injection, validation, REST endpoint exposure, and integration with the wider Spring ecosystem. FactoryFlow consequently benefits from **Spring Web** for HTTP communication, **Spring Security** for protected access, **Spring Data JPA** for persistence integration, and **framework support** for scheduling and email delivery. These capabilities are integrated within one coherent backend application rather than assembled as disconnected technical services.

The use of **Spring Boot** does not remove the architectural separation established for FactoryFlow. **Controllers** define the HTTP boundary, application services coordinate use cases, domain components represent business concepts and rules, while repositories and infrastructure adapters provide access to persistence and external systems. The resulting **modular monolith** combines straightforward deployment and testing with clear internal boundaries, allowing major capabilities such as deterministic interpretation, reporting automation, and Maintenance Intelligence to evolve without collapsing into a single tightly coupled implementation.

The following subsection examines how this foundation is organized into the **layered and modular architecture of the FactoryFlow backend**.

IV.2. Layered and Modular Backend Organization

FactoryFlow adopts a **modular-monolith architecture**: the backend is deployed as one Spring Boot application while remaining internally divided into explicit architectural layers and cohesive functional modules. Authentication, acquisition, deterministic interpretation, reporting, analytics, and notifications therefore belong to the same deployable system rather than to independent microservices. PostgreSQL, the private PaddleOCR runtime, generated-report storage, and SMTP remain outside this application boundary and are accessed through dedicated infrastructure adapters. **Figure 3.6** presents this organization.

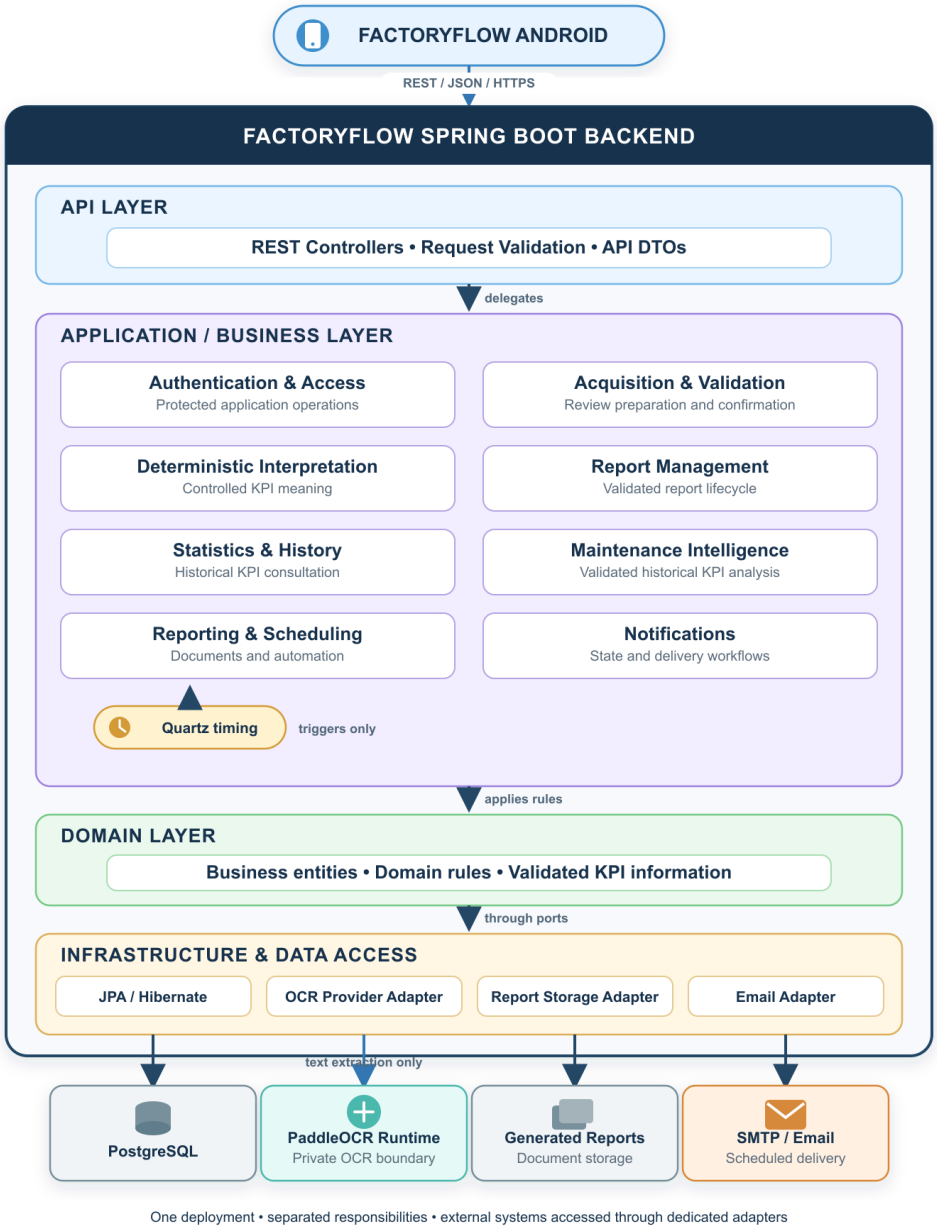

FIGURE 3.6 *Layered and modular architecture of the FactoryFlow Spring Boot backend*
Source: Created by the author using Figma (figma.com).

The **API layer** constitutes the authenticated HTTP boundary exposed to the Android application. REST controllers receive typed API DTOs, apply request validation, and delegate accepted operations to the application layer. Controllers do not determine how KPI values are interpreted, how documents are generated, or how analytical results are calculated. These responsibilities belong to application services organized around FactoryFlow capabilities, including Authentication and Access, Acquisition and Validation, Deterministic Interpretation, Report Management, Reporting and Scheduling, Statistics and History, Maintenance Intelligence, and Notifications. This separation also distinguishes acquisition workflow coordination from the deterministic mechanism that assigns business meaning to acquired information.

The **domain layer** preserves the business concepts and rules that remain meaningful independently of HTTP communication or database representation. It includes concepts such as KPIs, maintenance reports, validation states, schedules, notifications, generated-report metadata, and analytical results. Validated KPI information is especially important because conventional statistics and Maintenance Intelligence must operate on confirmed historical data rather than directly on unreviewed acquisition results. Keeping this meaning separate from controllers and persistence structures prevents infrastructure decisions from redefining business behaviour.

The **infrastructure and data-access layer** implements the technical boundaries required by the application. Spring Data JPA and Hibernate provide persistence access to PostgreSQL, while dedicated adapters connect the backend to the private PaddleOCR runtime, generated-report storage, and SMTP delivery. PaddleOCR remains responsible only for text extraction; deterministic interpretation stays inside the Spring Boot application. Quartz similarly supports Reporting and Scheduling by determining when an operation should begin, without owning report content, storage rules, period semantics, or email delivery.

This organization allows complex capabilities such as deterministic interpretation, reporting automation, and Maintenance Intelligence to coexist inside one backend without becoming one tightly coupled block of code. FactoryFlow consequently retains the deployment and testing simplicity of a single application while preserving replaceable infrastructure boundaries and maintainable business modules. The architecture establishes where each responsibility belongs; the following subsection explains how an individual request crosses these boundaries.

IV.3. Request Processing and Service Boundaries

The layered organization described previously becomes concrete when an operation reaches the FactoryFlow backend. For a conventional client request, processing begins at a **REST controller**, which receives the HTTP request, validates its API representation, and delegates the requested use case to the appropriate application service. The service coordinates the required domain rules and accesses persistence or external systems only through the corresponding repository or infrastructure abstraction. The result then returns through the controller as a structured API response to the Android application.

This processing path can be summarized as ***Android request*** $\rightarrow$ ***Controller*** $\rightarrow$ ***Application Service*** $\rightarrow$ ***Domain Rules / Repository or Adapter*** $\rightarrow$ ***Response***. However, FactoryFlow is not limited to simple CRUD operations. An acquisition request may require text extraction through the OCR provider followed by deterministic interpretation and preparation of review data; confirmation applies validation rules before information becomes trusted historical data; reporting operations may generate and store documents; and analytical requests may invoke Statistics or Maintenance Intelligence over validated KPI history. The application-service layer therefore acts as the **workflow coordination boundary** rather than as a direct bridge between controllers and PostgreSQL.

Not every backend workflow originates from an Android request. Scheduled operations can also be initiated internally when **Quartz** triggers a configured reporting task. In this case, the same application services remain responsible for period calculation, document generation, storage, notification, and optional email delivery. The trigger changes, but the business workflow remains inside the same controlled service boundaries. This avoids duplicating reporting logic between manually requested and automatically scheduled operations.

These boundaries preserve a clear distinction between **communication, business coordination, domain rules, and technical integration**. Controllers remain focused on the API contract, application services coordinate use cases, domain components preserve business meaning, and repositories or adapters isolate persistence and external technologies. The following sections examine these backend responsibilities in greater detail, beginning with the **data model and persistence architecture**.

V. Data Model and Persistence

V.1. PostgreSQL and Relational Persistence

FactoryFlow uses **PostgreSQL** as its central relational database because industrial maintenance information is structured, strongly connected, and expected to remain reliable over time. KPI measurements are associated with maintenance reports, generated documents originate from validated information, and schedules, notifications, and analytical operations depend on persistent application context. A relational model represents these relationships explicitly while providing transactions, constraints, referential integrity, and consistent querying. PostgreSQL also integrates naturally with the Spring Boot persistence ecosystem and provides a mature foundation for historical consultation, filtering, reporting, analytics, and continued system evolution.

The database preserves the information required to maintain FactoryFlow's operational and historical continuity. This includes KPI definitions and approved interpretation metadata, maintenance reports, individual measurements, acquisition and review state, generated-report metadata, schedules, notifications, and the confirmed historical information consumed by Statistics and **Maintenance Intelligence**. Validated KPI history therefore forms the persistent analytical foundation of the platform. Analytical outputs that require historical traceability, such as contextual alerts or retained analysis results, are persisted according to their application lifecycle. The objective is not to store every temporary technical value indiscriminately, but to preserve the information required for reliable workflows, traceability, document generation, analytics, and later consultation.

A crucial distinction is that **acquired or interpreted information does not automatically constitute trusted historical data**. Values obtained from pasted text, manual entry, shared content, or OCR must pass through the Review and confirmation workflow before becoming official. The persistence model must therefore preserve relevant lifecycle states and distinguish original input, interpreted information, user corrections, missing values, and confirmed results

instead of prematurely reducing them to a single final value. PostgreSQL remains exclusively behind the backend boundary: the **Android application never accesses the database directly**. Every persistent operation passes through the Spring Boot backend, where authentication, validation, and business rules determine what may be stored, updated, or returned.

V.2. Persistence with Spring Data JPA and Hibernate

FactoryFlow uses three complementary persistence technologies with distinct responsibilities. The **Jakarta Persistence API (JPA)** defines the standard annotations and contracts used to map Java persistence objects to relational structures. **Hibernate** implements this standard and performs the object–relational mapping operations required to translate entity state and relationships into SQL operations against PostgreSQL. **Spring Data JPA** builds on this foundation by providing repository abstractions that reduce repetitive data-access code. The resulting technical path can be summarized as ***JPA entity*** → ***JPA mapping*** → ***Hibernate*** → ***SQL/PostgreSQL***. This mapping reconciles the object-oriented representation used by the backend with the tables, columns, keys, and relationships used by the relational database.

FactoryFlow application services do not construct SQL statements or manage database connections directly. Instead, they depend on repository interfaces representing the persistent information required by each use case. These repositories support the retrieval and storage of maintenance reports, KPI measurements, validation information, schedules, notifications, generated-report metadata, and analytical results that require persistence. The complete access path is therefore ***Application Service*** → ***Repository*** → ***Spring Data JPA/Hibernate*** → ***PostgreSQL***. This structure reinforces the layered organization illustrated in **Figure 3.6** while preventing database-specific concerns from spreading into application workflows.

The repository boundary ensures that persistence technology supports the business model without defining its rules. A service confirming a maintenance report, **for example**, remains responsible for validation and confirmation behaviour rather than SQL syntax, connection handling, or entity-loading mechanics. Concepts such as missing measurements, validation status, user corrections, traceability, and trusted historical information retain their business meaning **independently** of their database representation. When an operation modifies several related persistent elements, Spring transaction management allows those changes to be treated as **one logical unit**, reducing the risk of partial updates leaving the stored state inconsistent.

Mapping Java objects to relational storage explains how information is persisted technically; the following subsection examines how FactoryFlow preserves its meaning and history through **Data Integrity and Traceability**.

V.3. Data Integrity and Traceability

Within FactoryFlow, **data integrity is not limited to database constraints or valid column types**. It also represents **semantic integrity**: preserving what maintenance information means, where it originated, and how trustworthy it is at each stage. A measurement entered manually,

extracted from an image, interpreted from operational text, corrected during Review, and explicitly confirmed does not possess the same trust status throughout its lifecycle. FactoryFlow therefore preserves this progression instead of treating every acquired value as immediately trusted. **Figure 3.7** illustrates this controlled lifecycle.

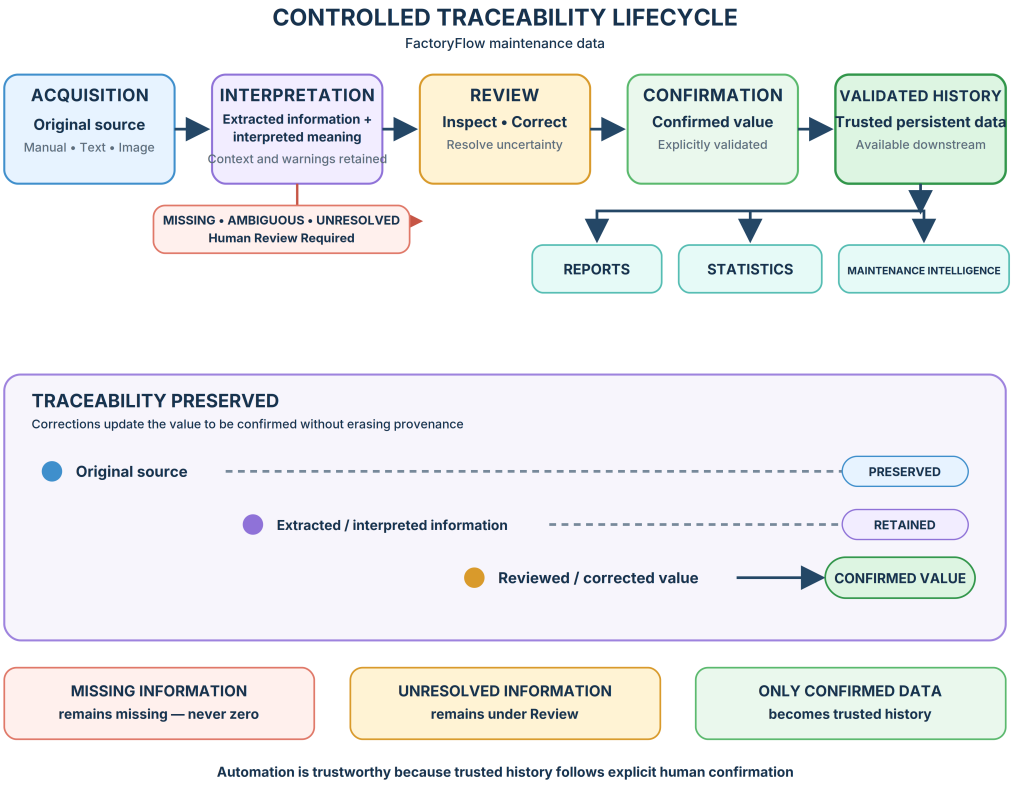

FIGURE 3.7 *Controlled traceability lifecycle of FactoryFlow maintenance data*
Source: Created by the author using Figma (figma.com).

The traceability model distinguishes the successive representations of maintenance information: **original source → extracted or interpreted information → reviewed or corrected value → confirmed value**. The original acquisition context remains preserved even when the Maintenance Engineer corrects an interpreted measurement. A correction changes the value that may subsequently be confirmed, but it **does not erase provenance**. Relevant interpretation context, including warnings, uncertainty information, and Review decisions, remains traceable where applicable without coupling persistence to the internal implementation of the parser. The system can therefore preserve both the accepted result and the controlled process through which it was obtained.

Several rules protect the transition into trusted history. **Missing information remains missing and is never converted automatically to zero**, because absence and a measured zero represent different industrial conditions. **Ambiguous or unresolved information remains under Review** until the Maintenance Engineer resolves or rejects it. Most importantly, acquired or interpreted information cannot become trusted historical data without **explicit confirmation**. The persistence model therefore distinguishes acquisition, Review, correction, and confirmation

states rather than relying on the mere existence of a database record. Only validated history, not raw acquisition output, provides the trusted foundation for generated reports, historical statistics, and Maintenance Intelligence.

Once confirmed, the resulting information can safely contribute to PDF and Excel reporting, KPI statistics, trend analysis, anomaly detection, forecasting, and contextual alerts because it has passed through a controlled human-validation lifecycle. FactoryFlow can therefore automate downstream exploitation while retaining accountability: **automation remains trustworthy because provenance is preserved and trusted history requires confirmation**. Preserving information integrity during application operation is one requirement; preserving the consistency of the database structure as FactoryFlow evolves is another. The following subsection therefore examines **Schema Evolution with Flyway**.

V.4. Schema Evolution with Flyway

FactoryFlow uses **Flyway** to manage PostgreSQL schema evolution through **ordered and versioned migration files**. Instead of applying informal modifications directly to a database, each structural change is described in a migration that becomes part of the backend source code and version history. Flyway executes pending migrations in their defined order and records the versions already applied in a dedicated schema-history table. The conceptual sequence is therefore ***migration files*** $\rightarrow$ ***ordered execution*** $\rightarrow$ ***schema version history*** $\rightarrow$ ***current PostgreSQL schema***. This process keeps the application's persistence expectations synchronized with the actual database structure and makes schema creation reproducible across development, testing, and deployment environments.

Throughout FactoryFlow's development, schema evolution accompanies capabilities requiring **tables, columns, constraints, indexes, or relationships** supporting reporting, scheduling, notifications, validation history, and Maintenance Intelligence. These modifications are introduced through new migrations rather than by rewriting previously applied database changes. A database environment can consequently be advanced to the expected schema state in a controlled manner without manually reproducing earlier modifications. The essential engineering principle is that **the database schema evolves under version control together with the backend instead of being modified informally**.

While Flyway preserves the **structural consistency** of persistent storage, access to that data and to protected backend capabilities must also be controlled. The following section therefore examines the **FactoryFlow Security Architecture**.

VI. Security Architecture

VI.1. Spring Security and Authentication Boundary

FactoryFlow uses **Spring Security** to establish the security boundary that protects access to backend capabilities before requests reach application services. Spring Security integrates with

the Spring web stack through a configurable chain of security filters that examines incoming requests and determines whether they may continue toward the REST controllers. This approach is appropriate for FactoryFlow because authentication and access control must remain centralized at the backend rather than being trusted to individual Android screens or application services.

The backend defines this behaviour through a **SecurityFilterChain**. FactoryFlow follows a **stateless REST architecture**: the authentication endpoint, `POST /api/auth/login`, remains publicly accessible so that a user can establish an authenticated session, while the remaining protected API operations require valid authentication. Traditional server-side form login is not used. Instead, authentication is based on **bearer tokens**, allowing each protected request to carry the information required for the backend to verify the caller independently.

For subsequent API calls, FactoryFlow installs a dedicated **JwtAuthenticationFilter** before Spring Security's `UsernamePasswordAuthenticationFilter`. The filter examines the `Authorization` header of incoming requests, validates the supplied bearer token, and establishes the authenticated security context required by downstream processing when the token is valid. Invalid or expired authentication information is therefore rejected before protected application functionality is reached. Once authentication has been established, controllers and application services can operate under a verified user identity rather than trusting identity information supplied directly by the Android interface.

This architecture creates a clear separation between **security enforcement and business processing**. Spring Security determines whether a protected request is allowed to cross the backend security boundary, while controllers and application services remain responsible for FactoryFlow workflows such as report management, validation, scheduling, and analytics. The Android application may control which authenticated screens are presented to the Maintenance Engineer, but **the backend remains the authoritative access-control boundary**. The following subsection examines how this security boundary is implemented through the **JWT-based authentication flow**, from credential verification and token issuance to authenticated REST requests.

VI.2. JWT Authentication and Secure Session Lifecycle

FactoryFlow implements authentication as a **stateless exchange between the Android application and the Spring Boot backend**. A JSON Web Token **(JWT)** is a compact signed representation through which the backend can associate subsequent requests with an authenticated identity. Unlike a traditional server-side session, the backend does not retain an HTTP session for each Android client. The interaction instead follows the sequence ***credentials establish identity once → backend issues a JWT → Android presents that JWT with every protected request***. A JWT contains a header, a payload, and a signature. In FactoryFlow, the token is **signed rather than encrypted**: its signature protects its integrity and authenticity, whereas its payload must not be considered confidential storage.

Authentication begins when the Maintenance Engineer submits an email address and password from the Android login screen. The endpoint `POST /api/auth/login` receives these credentials, and the `AuthController` delegates the operation to the `AuthenticationService`. Passwords are not compared with stored plaintext values. Instead, the submitted password is verified against the persisted **BCrypt password hash**. The `JwtTokenService` then issues an access token signed with the **HS256** algorithm. HS256 uses a **symmetric HMAC key** constructed from the protected `JWT_SECRET` configuration value, which FactoryFlow requires to contain at least 32 characters.

Figure 3.8 summarizes token issuance, secure Android storage, and authenticated access to protected backend operations.

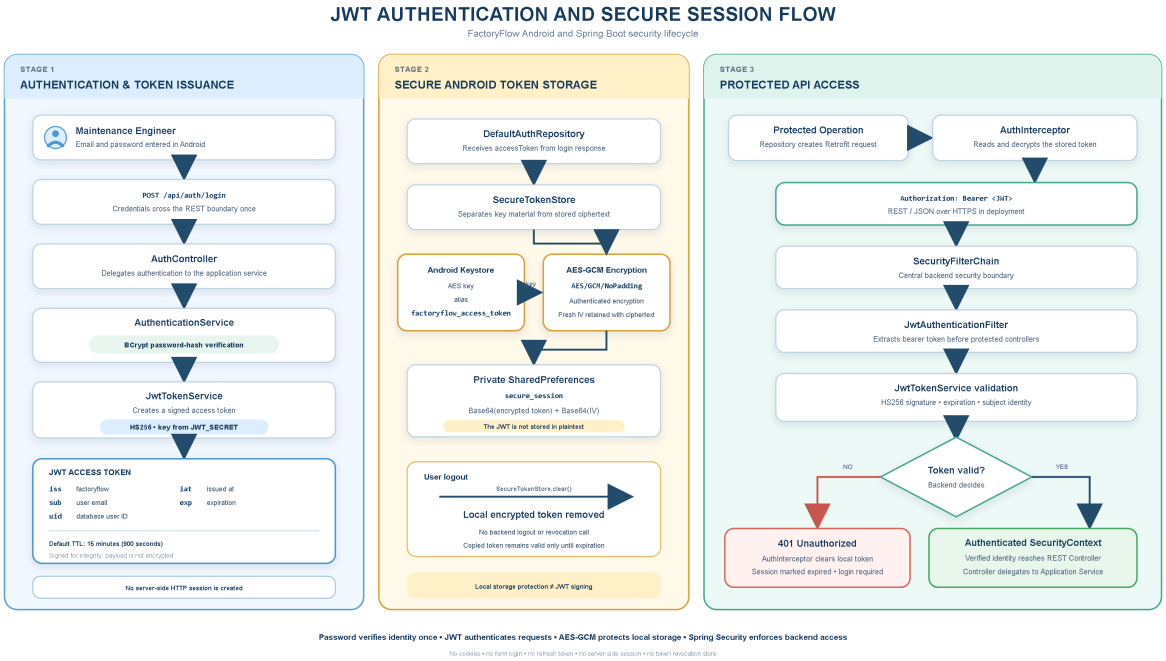

FIGURE 3.8 *JWT authentication and secure session flow in FactoryFlow*
Source: Created by the author.

The issued token contains the issuer `factoryflow`, the user's email address as its subject, the database user identifier in the `uid` claim, the issuance time in `iat`, and the expiration time in `exp`. The implemented token contains no role claim, token identifier (`jti`), or refresh-token information. FactoryFlow therefore uses a deliberately compact, short-lived access-token model rather than embedding additional authorization or token-management state in the JWT. The default validity period is **15 minutes**, equivalent to 900 seconds. It can be changed through `JWT_ACCESS_TOKEN_TTL`. The login response exposes the lifetime through `expiresInSeconds`. Nevertheless, the backend remains authoritative for token validity: a retained Android token is considered usable only while protected requests continue to be accepted by the server.

As illustrated in **Figure 3.8**, receiving a JWT is followed by a distinct **local-storage protection stage**. The `DefaultAuthRepository` passes the access token to `SecureTokenStore`.

The token is not written as plaintext to Android preferences. The `AES/GCM/NoPadding` transformation encrypts it instead. The associated AES key is retained by the **Android Keystore** under the alias `factoryflow_access_token`. The encrypted token and its initialization vector are Base64-encoded and stored in the application's private `secure_session` `SharedPreferences` file. This separation gives two mechanisms different responsibilities: the JWT signature protects the client–server authentication protocol, while AES-GCM and Android Keystore protect the token retained on the device.

For every subsequent protected operation, Retrofit creates the REST request and the Android `AuthInterceptor` retrieves the stored token through `SecureTokenStore`. After local decryption, the interceptor adds the token to the request using the `Authorization: Bearer` header. The Android application only presents this credential; it does not decide whether the token is valid. On the backend, the configured security filter chain receives the request first. The **`JwtAuthenticationFilter`** runs before protected controllers are reached. The token decoder verifies the HS256 signature and temporal validity. The `FactoryFlowUserDetailsService` then uses the extracted subject email to load the corresponding user account from the database. A valid result establishes an authenticated Spring Security context, after which the request may proceed to the REST controller and its application service.

A protected operation may return `HTTP 401 Unauthorized`. In that case, the Android `AuthInterceptor` clears the locally stored token and marks the session as expired. The authenticated application state therefore ends and the Maintenance Engineer must authenticate again. This behaviour also handles access-token expiration: once the backend refuses the expired credential, Android removes the unusable local session instead of continuing to present it.

FactoryFlow currently implements **local logout rather than server-side token revocation**. Logging out calls `SecureTokenStore.clear()`, removes the encrypted token and initialization vector from the Android client, and returns the application to its unauthenticated state. There is currently no backend logout endpoint, token blacklist, revocation store, refresh-token mechanism, or `jti` claim. Consequently, an independently copied token is not cryptographically invalidated by local logout and remains usable only until its configured expiration. FactoryFlow limits this exposure through its deliberately short default access-token lifetime while retaining a fully stateless backend authentication model.

VI.3. Security Responsibilities and Trust Boundaries

FactoryFlow's security architecture combines several controls whose responsibilities must remain distinct. **BCrypt** protects stored passwords through one-way hashing and is used only when credentials are verified during login. **HS256** protects the integrity and authenticity of JWT access tokens exchanged with the backend. **AES-GCM** protects a retained token against plaintext disclosure from Android application storage, while **Android Keystore** isolates the corresponding encryption key from the stored ciphertext. Because a signed JWT payload is not

encrypted, confidentiality for credentials and bearer tokens while crossing the network depends on using **HTTPS in the deployment environment**.

The Android interface manages authentication state and controls visible screens, but UI visibility is not an authorization mechanism. Every protected request must cross the Spring Security filter chain, and the backend remains responsible for validating the caller before exposing application capabilities. Controllers then delegate operations to application services, where input validation and business rules remain server-side. The Android client never connects directly to PostgreSQL; persistent information is accessible only through authenticated backend operations. Hiding a screen or action on Android therefore improves the user experience but cannot replace backend enforcement.

These boundaries create a defense-in-depth structure across the platform: Android securely retains and presents the credential, Spring Security verifies requests at the backend boundary, application services protect workflow rules, and PostgreSQL remains isolated behind the backend. The adopted model deliberately favors short-lived stateless access tokens over server-side revocation state. Refresh-token rotation or explicit token revocation represent possible security-hardening options for deployments requiring longer-lived sessions, but they are outside the implemented FactoryFlow authentication model.

VII. OCR and Acquisition Architecture

VII.1. Acquisition Channels and OCR Boundary

FactoryFlow deliberately supports three complementary acquisition channels because industrial maintenance information does not always reach the Maintenance Engineer in the same form: **Manual Entry**, **Paste Message**, and **Image Import**. Although these channels eventually converge toward the same controlled Review and confirmation lifecycle, **their processing routes are not identical**. Manual Entry already provides structured KPI values and therefore bypasses both OCR and textual interpretation. Paste Message provides machine-readable text and consequently bypasses OCR while entering deterministic interpretation directly. Image Import first requires text extraction before its result can be interpreted. The three routes can therefore be summarized as ***image*** $\rightarrow$ ***OCR*** $\rightarrow$ ***deterministic interpretation***, ***pasted text*** $\rightarrow$ ***deterministic interpretation***, and ***manual values*** $\rightarrow$ ***structured validation and draft workflow***.

Optical Character Recognition (OCR) converts visible textual content contained in an image into machine-readable text. This capability is particularly relevant to FactoryFlow because operational maintenance information may already exist in **WhatsApp screenshots or other imported images**. Requiring the Maintenance Engineer to transcribe this content manually would reproduce part of the repetitive workflow that FactoryFlow is intended to reduce. However, OCR has a deliberately restricted architectural responsibility. It identifies characters and textual lines, but it does not determine which KPI a line represents, whether a missing marker denotes an absent measurement, whether zero is a legitimate value, whether two observa-

tions are duplicates, whether a fuzzy suggestion is acceptable, or whether acquired information may become trusted. These decisions belong to FactoryFlow's deterministic interpretation and human-validation pipeline. The essential distinction is therefore that **OCR extraction ≠ business interpretation**.

For image-based acquisition, an image may originate from the Android gallery or arrive through an **Android Share Intent**. Both routes use the same acquisition workflow. The `OcrAcquisitionViewModel` coordinates the operation and delegates image access and transmission to `DefaultOcrRepository`. The repository reads the selected Android URI, accepts **JPEG, PNG, and WebP** content, and enforces a maximum upload size of **10 MiB**. The selected image is transmitted to `POST /api/ocr/recognize` as `multipart/form-data` using its binary image bytes rather than a Base64 representation. After receiving the request, the Spring Boot backend performs the MIME-type and size checks again before invoking OCR. This duplication is intentional: **backend correctness and security must not depend exclusively on validation performed by the Android client**.

Figure 3.9 presents the three acquisition routes and identifies the exact boundary between OCR extraction and FactoryFlow business interpretation.

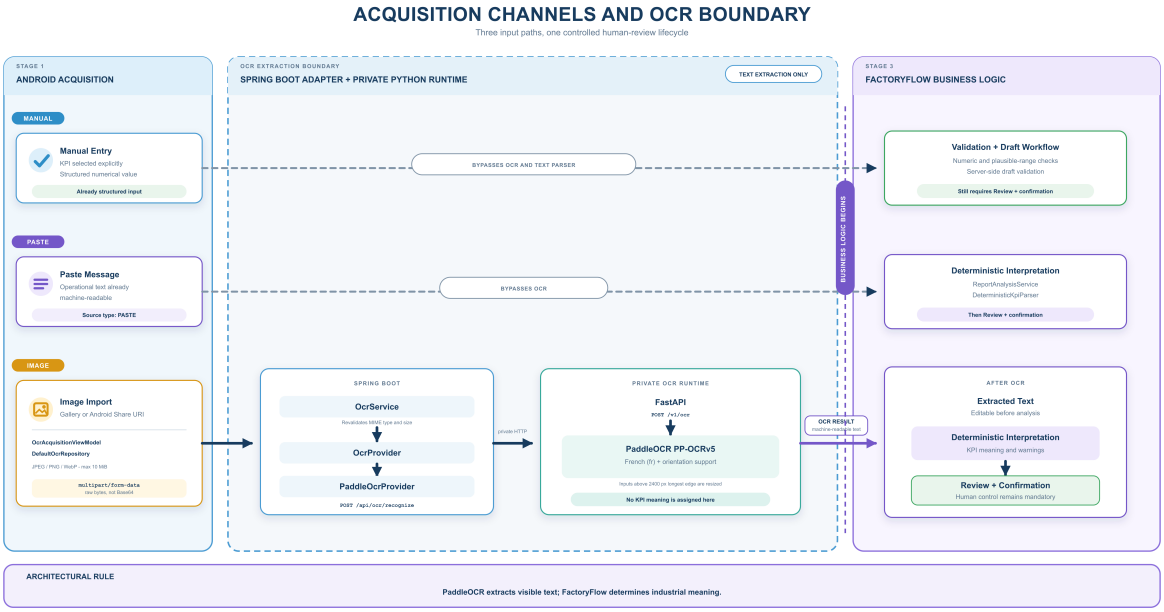

FIGURE 3.9 *Acquisition channels and OCR boundary in FactoryFlow*
Source: Created by the author.

As illustrated in **Figure 3.9**, the Spring Boot application does not embed PaddleOCR directly. The `OcrService` depends on the `OcrProvider` contract, whose current concrete adapter is `PaddleOcrProvider`. This adapter forwards the image through a private HTTP request to a separate Python runtime. The resulting sequence is ***Android → Spring Boot → OcrProvider → private PaddleOCR runtime***. In particular, **the Android application never communicates directly with PaddleOCR**. This provider boundary keeps the Spring application independent from Python-specific OCR internals, allows PaddleOCR to execute in its appropriate runtime

environment, and permits the OCR implementation or its configuration to evolve without redesigning the Android acquisition workflow or the backend application services.

The private runtime is implemented using **FastAPI**. It exposes the `POST /v1/ocr` endpoint, which is invoked by `PaddleOcrProvider`. The runtime executes **PaddleOCR PP-OCRv5 configured for French (`fr`)**. Document-orientation classification and text-line orientation are enabled to accommodate imported content that may not be perfectly normalized, whereas document unwarping is disabled. Images whose longest edge exceeds **2400 pixels** are resized before recognition to limit unnecessary processing cost. The runtime returns the **combined recognized text**, individual recognized lines, per-line confidence scores and bounding boxes, an overall confidence score, the engine identifier, processing duration, and warnings such as low confidence or image downscaling. The Android acquisition state consumes the combined text, overall confidence, engine information, and warnings; line-level bounding boxes remain part of the OCR response but are not retained in the current UI state.

These confidence values must also be interpreted within their proper boundary. **OCR confidence expresses confidence in character and text recognition, not confidence in the industrial meaning of the recognized information**. A line may be recognized with high confidence while still containing an unknown label, representing the wrong KPI, using an unexpected unit, or requiring human judgment. Confidence can therefore support Review, but **it cannot establish business truth or replace explicit confirmation**. Once the OCR response has been received, the extracted text remains editable in Android and is subsequently sent to the report-analysis workflow. At that point, **OCR has completed its responsibility**: FactoryFlow possesses machine-readable text, but that text has not yet been assigned reliable industrial meaning.

The **Paste Message** path enters deterministic interpretation without invoking OCR. The Android application submits the pasted content to `POST /api/reports/analyze` with the acquisition source identified as `PASTE`, after which `ReportAnalysisService` delegates the text to `DeterministicKpiParser` together with the active KPI definitions. The internal interpretation rules applied by this parser are examined in the following subsection.

Manual Entry bypasses both OCR and the deterministic text parser because the Maintenance Engineer explicitly selects the KPI definition and enters its numerical value in structured form. Nevertheless, **bypassing interpretation does not mean bypassing validation**. Android verifies that non-missing values can be represented numerically and compares them with the configured plausible range. A value outside that range produces an `OUTSIDE_PLAUSIBLE_RANGE` warning rather than being silently accepted or rejected. The structured request is then processed by the common server-side draft workflow, and the **common Review and confirmation safeguards remain applicable**. Manual Entry therefore bypasses heterogeneous-text interpretation, **not the trust lifecycle through which information becomes official**.

Traceability begins when the analyzed or structured information is used to create a server-side draft. For Paste Message, the submitted textual source is persisted as `MaintenanceReport.rawText`.

For OCR-based acquisition, FactoryFlow persists the **OCR-extracted text after any edits made in the acquisition interface** and records whether the source was `GALLERY_OCR` or `SHARE_OCR`. The original image bytes, Android URI, and PaddleOCR bounding boxes remain transient during recognition and are **not persisted by the current backend**. FactoryFlow therefore preserves the textual source that actually enters deterministic interpretation, but it does not claim permanent provenance for the imported image artifact itself. For Manual Entry, `rawText` is null because no textual source exists; provenance instead consists of the `MANUAL` source type, selected KPI definition, entered value, warnings, authenticated user, and report timestamps. Having established how information enters FactoryFlow and where OCR ends, the next subsection explains **how deterministic interpretation assigns structured industrial meaning to the resulting textual source**.

VII.2. Deterministic Interpretation Pipeline

The acquisition mechanisms presented in the previous subsection produce either structured manual values or a textual source obtained from Paste Message or OCR. However, recognizing characters does not by itself establish their industrial meaning. A source may contain KPI labels, numerical measurements, missing-value markers, percentages, WhatsApp metadata, OCR artefacts, duplicated observations, or previously unknown business information. Consequently, FactoryFlow introduces a deterministic interpretation layer between text acquisition and human validation. Its central principle is that **text recognition provides syntax, whereas deterministic interpretation assigns controlled business meaning**.

In FactoryFlow, **deterministic** means that the same source text, processed with the same active KPI definitions and configured rules, produces the same interpretation. The backend does not rely on opaque probabilistic interpretation for authoritative KPI extraction and does not allow uncertain information to become trusted data automatically. Instead, it applies explicit rules for normalization, source-line classification, segmentation, numerical interpretation, KPI matching, validation, and duplicate detection. This approach supports the **reproducibility, explainability, and traceability** required for industrial maintenance reporting.

Textual analysis enters the backend through `POST /api/reports/analyze`. The `ReportAnalysisCon` delegates the request to `ReportAnalysisService`, which retrieves the active KPI definitions and invokes `DeterministicKpiParser`. The parser receives both the complete `rawText` and its acquisition source. This separation is important: normalized representations are used for comparison, but the original submitted text and individual source lines remain available as evidence. **Normalization supports interpretation without rewriting provenance.**

The first processing stage applies safe structural normalization. FactoryFlow normalizes Unicode text using NFKC, standardizes line endings, converts non-breaking spaces and selected Unicode hyphen variants, and removes surrounding whitespace. Label matching additionally uses lowercase, accent-independent, punctuation-normalized, and compact representations. These working forms allow labels such as `EAU`, `Eau`, and spacing or punctuation variants

to be compared consistently. Nevertheless, the original label and source line are retained in the resulting candidate so that the Maintenance Engineer can compare the interpretation with the submitted evidence.

Each original line is then classified as **empty**, **header**, **WhatsApp metadata**, or **business content**. Recognized headers and metadata, including timestamps and known contextual lines, are returned as `ignoredLines` with their classification reason. Empty lines are skipped. Business-like content is never discarded merely because its KPI association is unknown. This creates a deliberate distinction between **safe contextual noise** and **unresolved KPI-like information**: the former may be excluded from active interpretation, whereas the latter must remain visible for human resolution.

Content lines pass through conservative segmentation. The parser first recognizes explicit separators such as colons, equals signs, and arrow forms, followed by a controlled whitespace-based label/value pattern. It also supports a restricted two-line form in which a recognized KPI label is immediately followed by a standalone value or missing marker. If no safe segmentation can be established, the line is preserved in `unrecognizedLines` instead of being silently removed or forcefully interpreted.

Figure 3.10 summarizes the complete transformation from preserved textual evidence to the structured projection presented for Review.

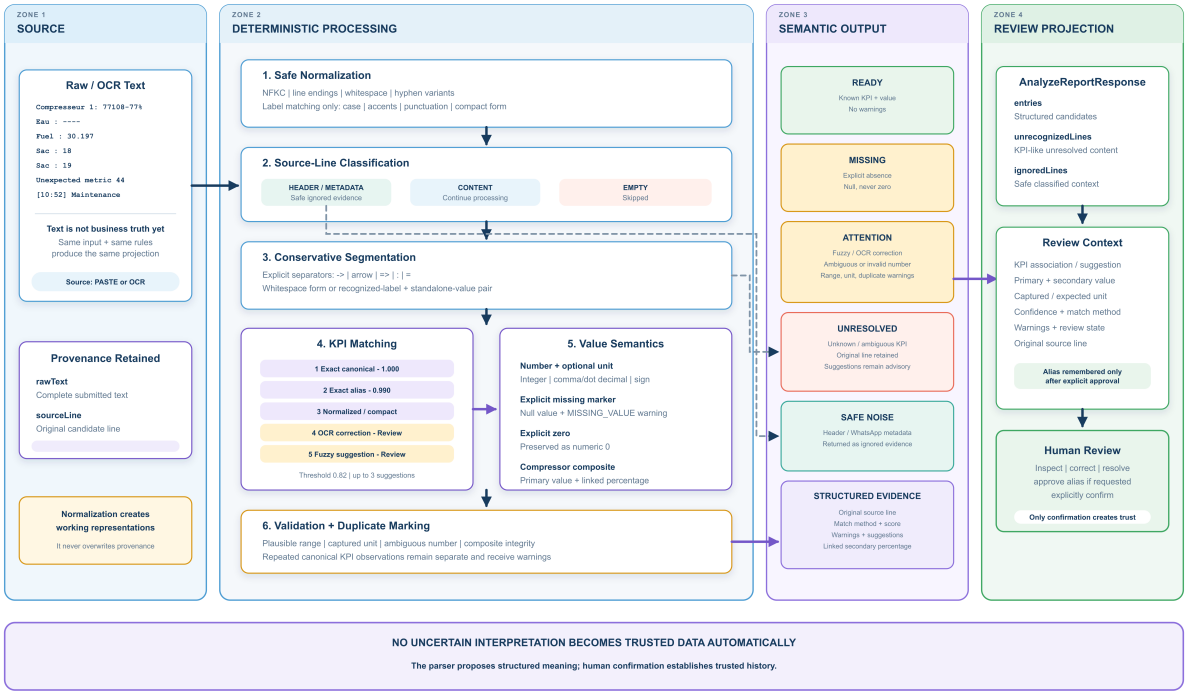

FIGURE 3.10 *Deterministic interpretation and Review-preparation pipeline in FactoryFlow*
Source: Created by the author.

Numerical interpretation distinguishes valid measurements, missing information, and invalid or ambiguous representations. FactoryFlow accepts signed integers and decimals, supports decimal commas and points, handles valid grouping separators, and captures an optional at-

tached unit. An explicit missing marker, including repeated hyphens, `N/A`, `NA`, `vide`, `manquant`, or `non renseigné`, produces a null extracted value with a `MISSING_VALUE` warning. It never becomes numerical zero. Conversely, an explicit `0` remains a valid measurement. This distinction is essential because **zero represents an observed value, while missing represents the absence of a usable observation**.

Some numerical forms cannot be interpreted safely from punctuation alone. For example, `30.197` may represent a decimal value or a grouped integer. When a single separator followed by three digits creates this ambiguity, the parser preserves an editable candidate and emits an ambiguity warning. A configured plausible range may resolve which interpretation is credible, but this resolution remains visible through a warning and therefore still reaches Review. FactoryFlow consequently avoids hiding numerical uncertainty behind an apparently precise value.

The value parser also supports the implemented compressor composite format. For KPI definitions whose code follows the compressor convention, a source such as `Compresseur 1: 77108-77%` is interpreted as a **primary value of 77108** associated with a **secondary percentage of 77%**. Both values remain linked to the same source observation. The parser separately validates the primary value, the secondary value, and the percentage range. This preserves the meaning of the original composite measurement instead of splitting it into unrelated KPI records.

KPI association follows an ordered authority hierarchy. FactoryFlow first attempts an **exact canonical match** with a score of 1.0000, followed by an **exact approved alias** with a score of 0.9900. Normalized and compact comparisons use scores of 0.9700 and 0.9500 respectively. A restricted OCR-aware comparison, including controlled substitutions such as `0` for `o` or `1` for `l`, uses a score of 0.9200 and explicitly requires Review. Only after these stronger mechanisms fail does the parser apply fuzzy similarity.

The fuzzy matcher combines normalized Levenshtein similarity with token-based Dice similarity. Its default acceptance threshold is 0.82, and at most three suggestions are returned. Candidates whose leading scores remain too close are treated as ambiguous instead of being resolved by an arbitrary highest-score choice. A fuzzy association therefore produces a `LOW_CONFIDENCE` warning and enters the `ATTENTION` state. **Fuzzy matching assists Review; it does not establish business truth.**

FactoryFlow also separates suggestions from reusable knowledge. A fuzzy or OCR-corrected association does not automatically create an alias. An alias is remembered only when the Maintenance Engineer explicitly approves the proposed association and requests that it be retained. Android then submits the decision to the KPI-definition alias endpoint, and the backend records the alias with the `USER_APPROVED` origin and authenticated approval information. This prevents one incorrect suggestion from silently modifying the interpretation of future reports.

After matching and value extraction, the parser applies validation rules. Captured units are compared with the expected KPI unit, numerical values are checked against configured plausible ranges, composite values are verified, and match-specific warnings are retained. If the same canonical KPI appears more than once, the observations remain separate and each

receives a DUPLICATE_KPI warning. FactoryFlow does not overwrite the first value with the second because repeated entries may represent an accidental duplicate, a correction, or distinct source observations requiring human judgement.

The resulting review projection uses four actual states. A known KPI with a valid value and no warnings becomes READY. An explicit absence becomes MISSING. Any recognized candidate carrying a correction, ambiguity, range, unit, composite, fuzzy-match, or duplicate warning becomes ATTENTION. A candidate for which no KPI definition can be established becomes UNRESOLVED. Composite values and duplicate observations are therefore **structured evidence or warning conditions**, not independent top-level review states. Safe headers and metadata are returned separately as ignored-source evidence.

The backend returns an AnalyzeReportResponse containing the original source and raw text, state counters, structured entries, unrecognized lines, and safely ignored lines. Each structured entry provides the original source label and line, matched or suggested KPI information, primary and optional secondary values, captured and expected units, match method, confidence, warnings, and review state. Android uses this projection to construct the editable draft presented to the Maintenance Engineer.

The deterministic parser therefore produces **explainable and traceable interpretation proposals**, not official maintenance history. Original evidence, uncertainty, warnings, unresolved content, and repeated observations survive processing so that they can be inspected rather than hidden. The governing boundary is explicit: **the parser proposes structured meaning; human confirmation establishes trusted history**. The following subsection examines how Review enforces this boundary and prevents uncertain information from becoming official without explicit validation.

VII.3. Review, Uncertainty and Human Validation

Deterministic interpretation produces structured observations, but structure alone does not make information authoritative. A candidate may still contain a warning, an explicit absence, an uncertain KPI association, a duplicate observation, or unresolved business content. FactoryFlow therefore implements Review as the **human-control layer between automated interpretation and trusted historical information**. The Android interface presents the available evidence and controlled resolution actions, while a persistent server-side draft keeps the information non-authoritative until the confirmation conditions are satisfied.

VII.3.1. Review States and Evidence

The analysis result is converted by Android from AnalyzeReportResponse into a DraftReportRequest and submitted to POST /api/reports/drafts. The backend persists a MaintenanceReport whose status remains DRAFT throughout Review. This makes Review a resumable business workflow rather than temporary Compose state. The engineer may leave the screen, reload the report, and continue from the persisted draft without promoting its values into official history.

The Android `ReviewViewModel` loads the persisted draft together with the active KPI definitions, while `ReviewScreen` organizes the observations into four semantic states: `READY`, `MISSING`, `ATTENTION`, and `UNRESOLVED`. These states describe what human action remains before confirmation; they are not persisted report statuses.

A `READY` observation has an assigned KPI, a usable interpreted value, and no warning that blocks confirmation. **Ready means eligible for confirmation, not already confirmed.** The current Ready card allows the engineer to inspect the result and its source evidence or remove the observation. It does not expose a value editor once the observation has reached this state.

A `MISSING` observation represents an explicit and understood absence of measurement. The engineer may preserve the absence, enter a corrective value, or remove the observation. An unchanged missing entry does not block confirmation and may retain a null final value. If the engineer enters a value, the entry becomes a corrected-missing case and requires explicit validation before it can become Ready. This distinction is fundamental: **missing is a valid semantic state, whereas unresolved means that the business meaning remains incomplete**.

An `ATTENTION` observation contains structured information, but automated interpretation alone is insufficient for authoritative acceptance. This state covers duplicate observations, numerical ambiguity, plausible-range and unit warnings, fuzzy or OCR-aware associations, and composite-integrity conditions. The interface allows the engineer to inspect the original source and warning, edit primary or secondary values where applicable, remove the observation, and explicitly validate it. Validation clears the blocking warning set from the current draft. **Attention does not mean incorrect; it means that the interpretation requires an accountable human decision.**

Duplicate KPI observations remain separate entries carrying `DUPLICATE_KPI`; neither value overwrites the other. Each occurrence must be validated independently because the parser cannot determine whether it represents repetition, correction, or a distinct observation. Composite compressor measurements remain one linked entry containing primary and secondary extracted, current, and final values. This preserves the relationship between the principal measurement and its associated percentage throughout Review and confirmation.

An `UNRESOLVED` parsed entry may be associated with its suggested KPI, assigned to another existing definition, used to create or reuse a detected KPI, edited, or removed. An unsegmentable but potentially business-relevant source line follows a narrower path: it may be assigned to an existing KPI or explicitly ignored. Direct KPI creation is not exposed for this unrecognized-line path. FactoryFlow therefore retains business-like content without pretending that every unknown line already has a reliable numerical interpretation.

Review remains evidence-based because the draft does not contain only a proposed final number. It retains the original source label and line, extracted and current values, assigned or suggested KPI, confidence and matching information, captured unit, warnings, unknown-line resolution, and linked secondary measurement where applicable. The complete `rawText` also remains available in the source panel. The source records what FactoryFlow received, the

extracted value records what automation initially interpreted, and the current value records the editable draft decision.

Figure 3.11 shows how these states and evidence pass through human resolution and the backend confirmation gate before becoming trusted history.

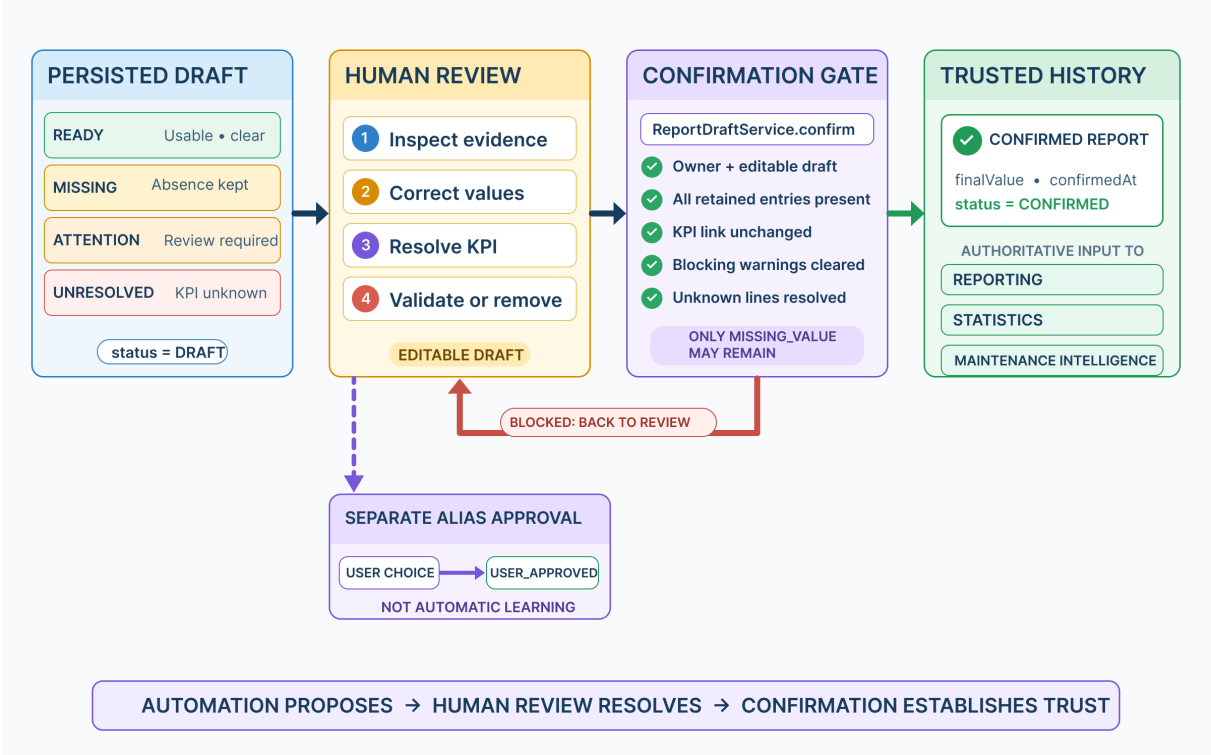

FIGURE 3.11 *Human Review, confirmation, and trust boundary in FactoryFlow*
Source: Created by the author.

VII.3.2. Corrections, Draft Continuity and Provenance

FactoryFlow separates the automated interpretation, editable decision, and authoritative result through three persisted values:

`extractedValue` ⟶ `currentValue` ⟶ `finalValue`

The same structure exists for the secondary component of a composite measurement. `extractedValue` records the parser or manual-entry candidate, `currentValue` records the editable draft state, and `finalValue` is assigned only during confirmation. For example, if the source contains `Eau : 45,7` and the engineer corrects the interpretation to `45.2`, FactoryFlow retains `45.7` as the extracted value and confirms `45.2` as the final value. **Correction changes the value eligible for confirmation; it does not rewrite the original evidence.**

The persisted `editedByUser` flag identifies entries whose value or KPI association has been changed. It becomes true when the primary or secondary current value differs from its extracted counterpart, when the engineer assigns a KPI, when a synthetic missing entry is added, or when a submitted final value differs from the extracted value. Acknowledging an unchanged

warning does not by itself necessarily mark the entry as edited; validation and modification remain distinct actions.

Draft continuity is implemented through authenticated creation, retrieval, replacement, entry-removal, unknown-line-resolution, and deletion endpoints. A complete draft update replaces the editable entry and unknown-line collections while preserving the report identity, source, and lifecycle. Report-level provenance includes the submitting user, submission time, last-update time, confirmation time, and optimistic-lock version. These fields support traceable state transitions, but the model should not be described as a complete per-edit audit ledger because it does not persist a separate historical record for every intermediate edit or warning acknowledgement.

VII.3.3. KPI Resolution and Explicit Knowledge Approval

Resolving an observation against an existing KPI modifies the current draft association. The engineer may accept the leading suggestion or select another active KPI definition. This action removes association-specific warnings such as unknown, ambiguous, low-confidence, or OCR-corrected matching conditions. It does not remove unrelated numerical, range, unit, duplicate, or composite warnings. **Resolving KPI identity does not automatically resolve every other data-quality concern.**

For a parsed unresolved entry, FactoryFlow also provides the detected-KPI create/reuse operation through `POST` at:

`/api/reports/{reportId}/draft/entries/{entryId}/add-kpi`

The backend first searches for an equivalent active definition. If one exists, it is reused. Otherwise, `KpiDefinitionService` creates an active definition from the detected label and captured unit, generates a normalized code, and assigns the result to the current draft entry. This is a focused resolution mechanism rather than a complete KPI-administration form; category, plausible ranges, and other configuration are not entered through this Review action.

Associating a label with a KPI for the current draft is separate from preserving that label as reusable parser knowledge. When the option is offered, the engineer must explicitly select *remember alias*. Android then sends a separate authenticated request to:

`POST /api/kpi-definitions/{id}/aliases`

The backend persists the alias and its normalized representation with the `USER_APPROVED` origin, approving user, and approval timestamp. This request is executed separately during Save or Confirm and is not an automatic consequence of association. **Accepting an association is not automatically the same as teaching the parser.** This separation prevents an uncertain or accidental mapping from silently changing future interpretation.

VII.3.4. Confirmation Gate and Trusted History

The normal Android workflow enables confirmation only when a report is loaded, at least one KPI entry exists, no Review entry remains blocking, and no unknown source line remains unresolved. This client-side gate guides the engineer and prevents incomplete actions from being presented as ready for submission. Nevertheless, the authoritative transition is enforced by the backend rather than by screen state.

Android submits the reviewed observations to `POST /api/reports/{id}/confirm`, where `ReportDraftController` delegates to the transactional `ReportDraftService.confirm(...)` operation. The service validates the request against the persisted draft. It verifies report ownership and editability, requires every retained draft entry to appear exactly once, rejects omitted or additional entry identities, prevents the client from changing the persisted KPI association during confirmation, checks the remaining warning state, and requires every unknown line to be assigned or ignored. **The Android interface guides Review; the backend controls the transition into confirmed history.**

In the implemented normal workflow, `MISSING_VALUE` is the only warning that may remain at confirmation. Duplicate, ambiguity, range, unit, fuzzy-match, OCR-correction, and composite-integrity warnings must first be explicitly reviewed and cleared. An accepted missing observation may therefore be confirmed with `finalValue = null`; confirmation does not require inventing a numerical value where the source supplied none.

When validation succeeds, each retained entry receives its `finalValue` and optional `secondaryFinalValue`. The report status changes from `DRAFT` to `CONFIRMED`, `confirmedAt` is recorded, and `editedByUser` reflects relevant differences between extracted and confirmed values. The backend then emits a `REPORT_CONFIRMED` notification.

A confirmed report is immutable through the normal workflow. Draft update, entry-removal, draft-deletion, and other editable operations reject a report whose status is already `CONFIRMED`; no confirmed-report correction endpoint currently exists. Confirmation therefore closes the Review lifecycle instead of creating another editable stage.

Only confirmed final values cross the trust boundary into downstream use. Dashboard queries, statistics, Excel and PDF generation, consolidated or scheduled reporting, and the current analytics services read reports whose status is `CONFIRMED` and use persisted final values rather than raw parser output. The same validated history forms the input boundary for the Maintenance Intelligence architecture presented later. **Confirmation is not merely the end of a mobile workflow; it authorizes maintenance information for downstream use as trusted historical evidence.**

VIII. Reporting and Scheduling Architecture

VIII.1. Reporting Architecture

FactoryFlow reporting begins only after human confirmation. The reporting layer does not consume OCR output, parser candidates, or editable draft values; it queries reports whose status is `CONFIRMED` and projects their `finalValue` and optional `secondaryFinalValue`. This boundary ensures that generated documents reproduce trusted maintenance history rather than provisional interpretation.

VIII.1.1. Individual and Consolidated Reporting

FactoryFlow supports two reporting scopes through the same `GeneratedReportService`. An individual export starts from one explicitly selected maintenance report. The backend loads that exact report, rejects it unless it is confirmed, and preserves its identity as the sole reporting source. **An individual export is therefore not equivalent to a one-day consolidated report**: it represents one validated source report.

A consolidated export combines confirmed reports belonging to an inclusive date interval. The implemented reporting types are `DAILY`, `WEEKLY`, `MONTHLY`, and `CUSTOM`. These types change the reporting scope and period validation, but not the underlying generation architecture. Consolidated generation also prepares an equal-length preceding period for comparative KPI analysis, whereas individual generation has no comparison period.

Reporting preserves the distinction between absence and zero. A confirmed null value remains missing and is excluded from ordinary numerical aggregates, while an explicit numerical zero remains a valid observation. Consequently, **missing information is never converted into zero merely to simplify document generation**.

VIII.1.2. PDF and Excel Generation

After resolving the reporting scope, `GeneratedReportService` constructs a common `ReportGenerationDat` projection containing the period, source reports, confirmed measurements, data-quality information, and traceability metadata. Format-specific generators consume this projection, allowing PDF and Excel output to share the same trusted source while adapting their presentation to different uses.

PDF generation is implemented by `PdfReportGenerator` using Apache PDFBox. It produces an A4 document containing report identification and period information, a reporting summary, KPI analysis, confirmed measurement details, data-quality indicators, traceability information, branding, and pagination. For weekly and monthly reports, a trend chart may be included when sufficient valid observations and page space exist. The generator selects one eligible KPI with at least two points rather than combining unrelated indicators or incompatible units in a single chart. PDF is therefore treated as a human-readable reporting artifact rather than a direct database export.

Excel generation is implemented by `ExcelReportGenerator` using Apache POI and `XSSFWorkbook`. Each workbook contains exactly one worksheet named `Rapport`. This work-

sheet organizes the report summary, KPI analysis, confirmed measurement details, data quality, traceability, and generation information into distinct sections. Dates and measurements are written as actual Excel date and numeric cells, preserving their usefulness for subsequent calculations. The current workbook contains no Excel charts.

Both generators preserve the same missing-value semantics but adapt their representation to the destination format. PDF renders a missing measurement as *Non renseigné*; Excel leaves the corresponding numeric cell blank and explains that blank cells represent unreported values. This prevents textual placeholders from contaminating numeric Excel columns while retaining the distinction between missing information and zero.

Figure 3.12 summarizes the complete path from confirmed history to format-specific generation, persistent document storage, and controlled Android retrieval.

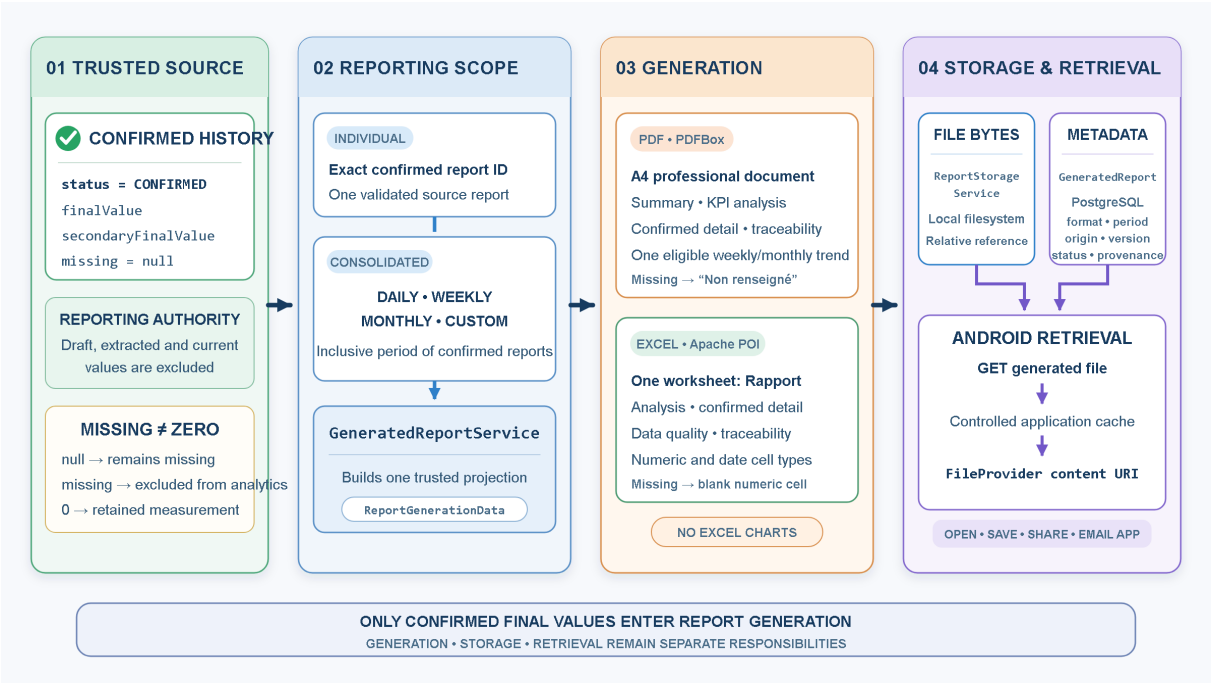

FIGURE 3.12 *Reporting generation, storage, and retrieval architecture in FactoryFlow*
Source: Created by the author.

VIII.1.3. Document Storage, Retrieval and Manual Sharing

Document generation is separated from storage through the `ReportStorageService` interface, which exposes storage, reading, and deletion operations. Its current implementation, `LocalReportStorageSer` writes the generated bytes to a temporary file and then moves that file atomically into the configured local storage root. The service returns a relative storage reference, keeping filesystem mechanics outside the PDF and Excel generators.

The document bytes and their application metadata have different persistence responsibilities. The binary PDF or Excel file resides in the configured filesystem, while PostgreSQL stores a `GeneratedReport` entity containing its format, reporting period, file name, relative storage reference, generation origin, generating user when applicable, generation timestamp, version,

source reports, and lifecycle statuses. **File content belongs to document storage; document identity, provenance, and lifecycle belong to persistent application metadata.**

This separation also supports failure isolation. If file storage succeeds but metadata persistence fails, the backend performs compensating deletion of the stored file. Conversely, a later distribution failure does not delete a successfully generated document. Generation, persistence, and distribution therefore remain distinct lifecycle stages.

Android retrieves a generated document through `GET/api/generated-reports/{id}/file` and stores the response temporarily under the application's controlled `generated-reports` cache directory. It then exposes the cached file through Android `FileProvider`, producing a temporary `content://` URI instead of sharing a backend or local filesystem path. From the generated-document interface, the user may open the file, save it to a selected location, share it through another application, or address it through an installed email application. These are user-initiated Android actions; unattended backend email delivery belongs to the scheduling architecture examined in the following subsection.

VIII.2. Scheduling and Automated Distribution

FactoryFlow separates scheduling infrastructure from reporting business logic. Quartz determines when an execution begins, while FactoryFlow application services determine the reporting period, generate the requested documents, store successful results, and coordinate optional distribution. **Quartz is therefore the runtime timing mechanism, not the reporting engine.**

VIII.2.1. Scheduling and Reporting Periods

A persisted `ReportSchedule` defines the recurrence, execution time, requested PDF and Excel formats, optional email distribution, recipients, and activation state. Scheduled reporting supports `DAILY`, `WEEKLY`, and `MONTHLY` recurrence; individual and custom exports remain user-initiated. All schedules use the `Africa/Casablanca` business timezone.

PostgreSQL is the persistent source of schedule configuration, whereas Quartz uses an in-memory runtime job store. The `QuartzScheduleCoordinator` translates each enabled schedule into a cron trigger and reconstructs these runtime definitions from PostgreSQL when the backend starts. When a trigger fires, `FactoryFlowReportJob` delegates the scheduled identifier and planned execution time to `ScheduleExecutionService`. This keeps reporting rules outside the Quartz job.

Scheduled generation targets the most recent completed period rather than the incomplete current period. `SchedulePeriodCalculator` maps a daily execution to the previous calendar day, a weekly execution to the previous completed Monday–Sunday week, and a monthly execution to the previous complete calendar month. The resulting inclusive period is then evaluated against confirmed maintenance history. **Scheduling determines when processing begins; period calculation determines which trusted data belongs to that execution.**

Figure 3.13 presents the complete sequence from persisted schedule configuration to durable execution feedback.

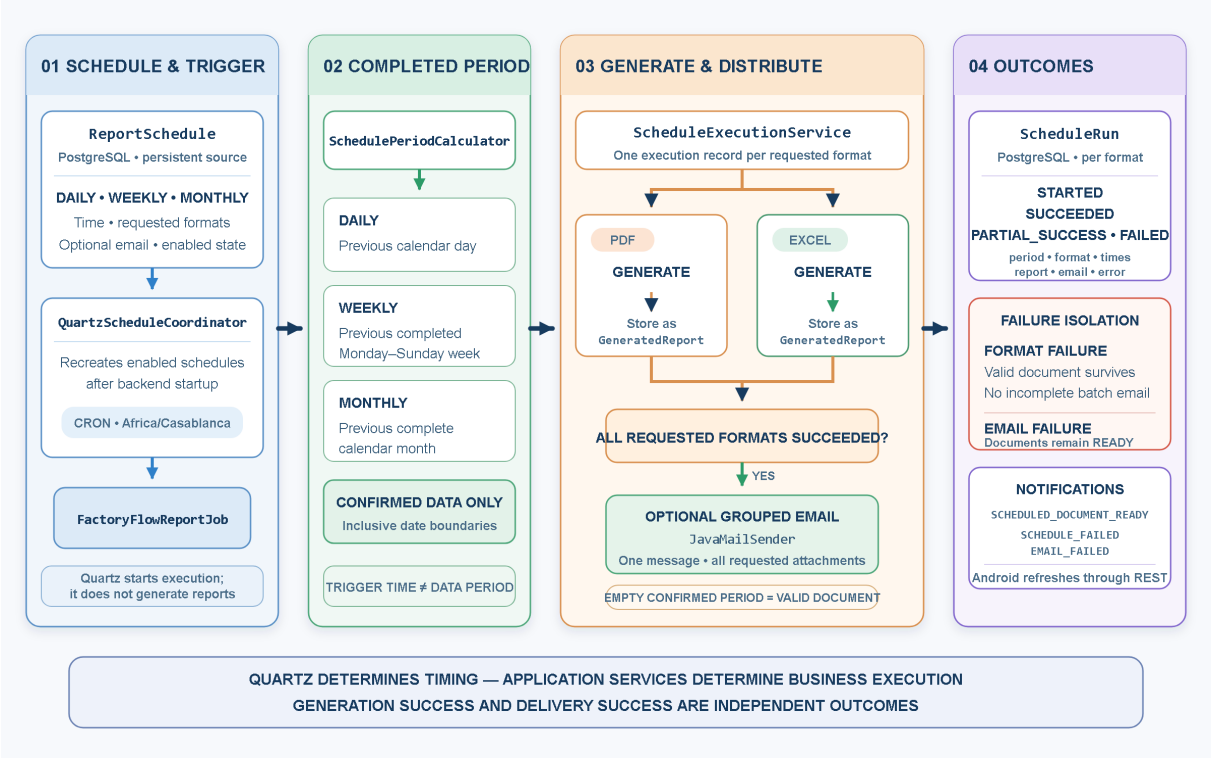

FIGURE 3.13 *Scheduled reporting, distribution, and execution-traceability architecture in FactoryFlow*

Source: Created by the author.

VIII.2.2. Automated Generation and Email Distribution

For each requested format, `ScheduleExecutionService` creates a separate `ScheduleRun` and invokes the reporting architecture established in the previous subsection. PDF and Excel are generated independently from confirmed final values, and each successful document is stored as a `GeneratedReport`. A period containing no confirmed observations remains a valid reporting outcome: FactoryFlow produces an explicit empty report instead of treating the absence of data as a technical failure.

When every requested format has been generated successfully and email distribution is enabled, the backend sends one grouped multipart message through `JavaMailSender`. The message contains plain-text and HTML representations together with all requested PDF and Excel attachments. Email is attempted only after document generation and storage; if one requested format fails, FactoryFlow suppresses the incomplete attachment batch.

A `DELIVERED` status means that the backend mail operation completed without an exception. It does not prove final mailbox delivery, user access, or the absence of a later rejection because FactoryFlow does not implement external delivery receipts. This distinction prevents infrastructure acknowledgement from being presented as stronger evidence than the implementation provides.

VIII.2.3. Execution Outcomes and Notifications

Each per-format `ScheduleRun` records the reporting period, format, scheduled and actual execution times, associated generated document, execution status, email status, and failure information. The meaningful execution states are `STARTED`, `SUCCEEDED`, `PARTIAL_SUCCESS`, and `FAILED`. State transitions are persisted independently so that a successful intermediate result is not rolled back by a later failure.

If one requested format fails, the failed run records the generation error while every successfully generated document remains available. The corresponding successful run becomes `PARTIAL_SUCCESS`, and no incomplete grouped email is sent. If all documents are generated but email delivery fails, the documents remain `READY`, their email status becomes `FAILED`, and the associated runs also become `PARTIAL_SUCCESS`. **A distribution failure cannot retroactively invalidate a document that was generated and stored successfully.**

FactoryFlow persists in-app notifications for outcomes such as `SCHEDULED_DOCUMENT_READY`, `SCHEDULE_FAILED`, and `EMAIL_FAILED`. Android retrieves these notifications through the authenticated REST API and may mark them as read. They therefore provide durable execution feedback rather than real-time push delivery.

IX. Maintenance Intelligence Architecture

X. Deployment and Integration Architecture

XI. UML Class Modeling

XII. Conclusion

CHAPTER 4

FACTORYFLOW IMPLEMENTATION

I. Introduction

II. Authentication and Access

III. Data Acquisition

IV. OCR Processing

V. Deterministic Parsing

VI. Human Review and Validation

VII. Manual Entry

VIII. Dashboard and Statistics

IX. PDF and Excel Reporting

X. Scheduling and Notifications

XI. Maintenance Intelligence Implementation

XII. Development Environment and Engineering Toolchain

XIII. Conclusion

CHAPTER 5

COMPLEMENTARY PROJECT: DOSAGE ANALYSIS

I. Introduction

II. Industrial Problem

III. Objectives

IV. System Architecture

V. Anomaly and Tolerance Management

VI. Dashboard and Real-Time Monitoring

VII. Technology Stack

VIII. Complementarity with FactoryFlow

IX. Conclusion

CHAPTER 6

VERIFICATION, TESTING AND RESULTS

I. Introduction

II. Testing Strategy

III. Unit Testing

IV. Integration Testing

V. OCR Validation

VI. Report Generation Validation

VII. Maintenance Intelligence Validation

VIII. End-to-End Testing

IX. Real-Device and User Validation

X. Results and Discussion

XI. Conclusion

GENERAL CONCLUSION AND PERSPECTIVES

REFERENCES

CHAPTER A

ADDITIONAL TECHNICAL MATERIAL